DAM/DIF

Optimisation de l'utilisation des caches L2/L3 et meilleure distribution des pages : prototypage d'un module noyau Linux

Sébastien Valat

Master II Informatique et Systèmes coopératifs.
Université de Savoie

Stage de fin d'études
du 1er juin au 30 septembre 2009

Tutoré au CEA/DAM Île-de-France par M. Marc Pérache
Suivi à l'université de Savoie par Christian Martel

Résumé

Sous Linux, une perte de performance du cache L2 a été mise en évidence sur la gamme Core 2 d'Intel alors qu'OpenSolaris et FreeBSD prennent en compte ce problème. Une solution actuelle sous Linux consiste à utiliser les *huge pages*, développées à l'origine pour les bases de données. Dans le cadre du HPC, il est toutefois difficile de se satisfaire des contraintes associées à cette implémentation. Ce rapport décrit donc l'implémentation d'un système de grosses pages adapté au contexte HPC sous Linux : VHP (Virtual Huge Page) sous la forme d'un module noyau.

Ce rapport débute par quelques rappels sur les caches et la gestion de la mémoire dans les systèmes d'exploitation, puis décrit l'implémentation de la solution retenue. Enfin nous détaillons les résultats obtenus sur différentes applications. Des gains de 6% ont pu être observés sur certaines tailles de problèmes avec *Linpack* ou encore 0,9% sur *bzip2*. Au cours des tests sur des applications plus concrètes nous avons toutefois observé une grande variabilité des résultats en fonction de l'allocateur utilisé.

Remerciements

Je tiens à remercier tout particulièrement mon tuteur de stage, Marc Pérache, pour l'aide et les conseils apportés tout au long de mon travail au CEA/DAM Île-de-France. Je tiens également à remercier spécialement François Diakhate, le doctorant avec qui j'ai partagé la chaleur des bureaux du bâtiment L1 Ocre. Je le remercie aussi pour les conseils qu'il a pu me fournir quant à l'implémentation de mon module noyau. Merci à Stéphane Zuckerman, doctorant à l'université de Versailles pour les échanges que l'on a pu avoir.

Merci à Hervé Jourdren pour son accueil sympathique lors de mon arrivée au CEA et pour ses visites régulières.

Un grand merci à l'équipe de développement de *VirtualBox* pour la création de cet outil magnifique nous évitant les mille redémarrages matériels nécessaires à la prise en main du développement noyau.

Je tiens à remercier Patrick Allal et Isabelle Visotto qui ont permis le bon déroulement de ce stage au niveau matériel et administratif.

Je remercie enfin tous les stagiaires de *l'open-space* : Carole, Clément, Emmanuel, Guillaume, Joseph, Kevin, Nicolas et Victor pour leur bonne humeur. Un grand merci aux doctorants de l'*espace nord* : François, Gaël et Marc, pour leur accueil chaleureux.

Table des matières

Introduction 1

1. Le Commissariat à l'Énergie Atomique 2

2. Rappels et état des lieux 5
2.1. Les caches processeurs . . . 5
2.2. Gestion de la mémoire dans un système d'exploitation . . . 6
2.3. Interaction de la pagination avec les caches . . . 9
2.4. Mise en évidence des caches . . . 9
2.5. Les *huge pages* sous Linux . . . 11
2.6. Pagination dans les autres OS . . . 13

3. Notre solution : Virtual Huge Pages 15
3.1. Objectifs et contraintes . . . 15
3.2. Description générale de la solution retenue . . . 16
3.3. Interface espace utilisateur / espace noyau . . . 16
3.4. Structures de données du module . . . 18
3.5. Fragmentation de l'allocation . . . 20
3.6. Support de l'allocation au premier accès . . . 20
3.7. Support des appels à *munmap* . . . 21
3.8. Support des appels à *mremap* . . . 22
3.9. Support des appels à *forks* . . . 22
3.10. Support d'un mode coloration de pages . . . 24
3.11. Support du swap . . . 25
3.12. Intégration dans les programmes cibles . . . 25
3.13. Multiplication des VMA et appel à *ioctl* . . . 26
3.14. Configuration et suivi du module . . . 27

4. Résultats 28
4.1. Boucles sur un tableau . . . 28
4.2. Temps d'allocation . . . 29
4.3. Linpack . . . 29
4.4. Applications à usage réel . . . 31
4.5. Sensibilité aux allocateurs . . . 33

Conclusion et ouverture 34

Annexes 35

A. Correctif de l'allocateur MPC 36

B. Configuration du Linpack 37

C. Résultats du Linpack 38

D. Configuration du code de MHD 39

Introduction

Au fil des années, le fossé entre les performances des processeurs et des mémoires s'est creusé de manière continue[1]. Tous les processeurs modernes utilisent les caches pour compenser en partie cet écart, et fournir un accès plus rapide aux données fréquemment utilisées par les applications. L'utilisation de ces mémoires très rapides intégrées au sein du processeur a toutefois un prix non négligeable qui est en général le facteur limitant leur taille. On trouve ainsi chez Intel des gammes de processeurs "pro" pour lesquelles la différence avec les modèles de base est en partie la taille des caches.

Dans le cadre des applications scientifiques utilisant de grosses quantités de données sur des calculs lourds, l'impact de ces caches sur le temps d'exécution peut devenir crucial et le choix de processeurs plus performants sur ce point impacte le prix des calculateurs conçus pour ces applications. Sous Linux, il a été mis en évidence une perte d'efficacité du cache L2 sur la gamme Core 2 d'Intel.

Sous Linux ce problème peut se régler par l'utilisation des *huge pages*, système qui utilise des pages de tailles plus importantes pour réduire la pression sur les caches et sur les TLB. [1] Ce système n'est toutefois pas utilisable en production dans un contexte HPC. [2] Ce rapport décrit une approche permettant de contourner les problèmes posés par les *huge pages*.

Ce rapport s'articule en trois parties : dans un premier temps nous ferons quelques rappels sur le fonctionnement des caches, la gestion mémoire et ferons un bilan des techniques utilisées dans OpenSolaris et FreeBSD pour limiter leur impact sur les caches. Nous traiterons ensuite de la description et de l'implémentation de la méthode proposée pendant ce stage. Nous terminerons ce rapport par quelques tests réalisés sur diverses applications pour évaluer l'impact du module.

1. *Translation Lookaside Buffer* : il s'agit d'un cache du processeur utilisé pour accélérer les traductions d'adresses virtuelles en adresses physiques.
2. Calculs Hautes Performances

1. Le Commissariat à l'Énergie Atomique

Le CEA [2], ou Commissariat à l'Énergie Atomique, est un ensemble de centres d'étude et d'expérimentation nucléaire fondé dans les années 50. Créé après la Seconde Guerre mondiale, son but premier était de fournir à la France les compétences nécessaires dans le domaine de l'Énergie et de la force de frappe nucléaire pour la Défense Nationale.

Depuis, ses objectifs ont été actualisés. Le CEA est maintenant un pôle important de recherche technologique dans différents domaines : l'énergie, les technologies pour l'information, la santé et la défense. Il a pour but de devenir le premier organisme européen de recherche technologique et garantir la pérennité de la dissuasion nucléaire.

Il dispose d'une culture croisée entre ingénieurs et chercheurs qui lui permettent de placer ses activités de la recherche fondamentale à la conception de projets industriels — tels que les réacteurs nucléaires par exemple — mais aussi de moyens importants comme des supercalculateurs, des réacteurs de recherche, des grands instruments de la physique, des lasers de puissance, ...

Implanté dans 9 centres répartis dans toute la France (cf. Figure 1.1), le CEA dispose de liens forts avec aussi bien les collectivités locales, que les universités ou les organismes de recherche, jusqu'à un niveau européen.

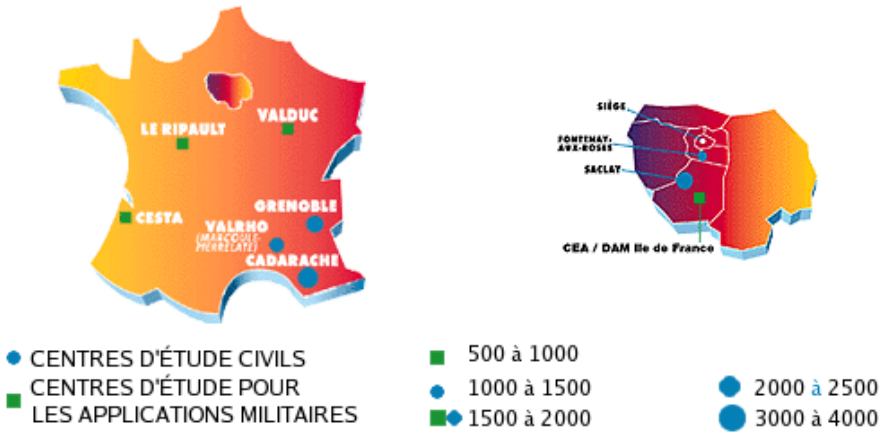

FIGURE 1.1.: Nature, taille et implantation des centres du CEA en France

Le CEA compte actuellement environ 15 000 employés pour un budget de 3,4 milliards d'euros.

La Direction des Applications Militaires

Le CEA/DAM, ou encore Direction des Applications Militaires, est le pôle responsable de la partie Défense du CEA. Ce pôle a le rôle de concevoir, fabriquer et entretenir les têtes nucléaires des forces militaires françaises. Apportant aux pouvoirs publics la garantie que ces têtes sont fiables et sûres, il est l'un des principaux artisans de la crédibilité de la dissuasion nucléaire française.

Aujourd'hui, l'objectif du pôle Défense est de continuer à assurer sur le long terme cette capacité de dissuasion sans recourir aux essais nucléaires, définitivement arrêtés depuis 1996. À cette fin,

sa priorité est de poursuivre le programme SIMULATION.

Ses missions portent également sur l'approvisionnement en matières nucléaires pour les besoins de la défense, les réacteurs nucléaires pour la propulsion des sous-marins et porte-avions, ainsi que sur la surveillance du respect des traités internationaux.

Le pôle Défense publie bon nombre de ses résultats scientifiques et développe des collaborations avec d'autres acteurs de la Recherche. Il valorise ses activités auprès des industriels par le transfert de technologies et le dépôt de brevets. Il s'est également engagé dans une importante démarche de certification qualité de l'ensemble de ses activités liées aux armes nucléaires.

Le pôle Défense compte aujourd'hui 4 500 collaborateurs, menant des activités réparties entre la recherche de base, le développement et la fabrication.

Il est implanté sur quatre centres :
- Valduc, en Bourgogne
- Le Ripault, en Touraine
- le Cesta, en Aquitaine
- DAM - Île-de-France (DIF)

Le centre DAM - Île-de-France

Le centre CEA/DAM - Île-de-France (DIF) a été créé en 1955, sur les communes de Bruyères le Châtel et d'Ollainville (cf. Figure 1.2), près d'Arpajon, à 35 km au sud de Paris, afin d'abriter les premières études de métallurgie des matières nucléaires utilisées dans les armes.
Aujourd'hui, le centre, auquel est rattaché le site de Moronvilliers près de Reims, est un grand établissement de recherche qui abrite l'un des plus grands centres de calculs d'Europe et des moyens de laboratoire originaux. Le stage s'est déroulé dans ce centre.

FIGURE 1.2.: Le centre DIF de Bruyères-le-Châtel

Depuis 2003, le centre DAM-Île-de-France héberge le complexe de calcul scientifique du CEA, qui regroupe l'ensemble des supercalculateurs du CEA. Il comprend l'ordinateur TERA-10 pour les besoins Défense, et les ordinateurs du Centre de Calculs pour la Recherche et la Technologie (CCRT), ouverts à la communauté civile de la recherche et de l'industrie.

Les prochaines avancées se tournent vers TERA-100, supercalculateur successeur de TERA-10. Le but est de construire un système petaflopique opérationnel en 2010, c'est-à-dire qui sera capable d'effectuer un million de milliards d'opérations par seconde.

Dans le cadre de la recherche européenne, le CEA participe au projet PRACE (*Partnership for Advanced Computing in Europe*) au travers du TGCC (Très Grand Centre de Calcul), qui hébergera des machines visant à créer une grille de calcul dont les différents membres mettront en commun des supercalculateurs, chacun disposant d'une puissance supérieure ou égale au petaflop.

Département des Sciences de la Simulation et de l'Information

Chaque centre est composé de différents départements. Le Département des Sciences de la Simulation et de l'Information est le département qui se consacre aux systèmes d'information et à la simulation numérique. Il regroupe l'ensemble des moyens informatiques du centre. C'est dans ce département que j'ai effectué mon stage.

2. Rappels et état des lieux

Afin de bien comprendre le contexte technique dans lequel doit s'intégrer notre travail, nous allons faire ici quelques rappels sur le fonctionnement des caches ; puis décrire la gestion de la mémoire dans les systèmes d'exploitation modernes. Nous allons ensuite essayer de comprendre la relation qui existe entre les deux pour mettre en évidence les raisons de pertes de performances observées sous Linux. Partant de ce constat nous ferons un bilan des techniques actuellement disponibles pour contourner ce problème sous Linux, FreeBSD et OpenSolaris.

2.1. Les caches processeurs

La technique des caches processeurs part du principe de cohérence spatiale et temporelle des données utilisées par un programme. À un instant donné, les programmes ont en effet tendance à accéder à de petits segments mémoires de manière répétitive. Les caches offrent donc la possibilité de placer temporairement ces segments dans une petite mémoire proche du processeur avec une latence d'accès plus faible que la mémoire centrale. Les données sont copiées dans le cache en groupe, en supposant que les informations voisines seront utilisées dans un futur proche. L'application dispose donc de l'équivalent d'une mémoire plus rapide et pourra utiliser le processeur de manière plus efficace.

2.1.1. L1/L2/L3 sur les processeurs modernes

Les applications modernes utilisant de plus en plus de mémoire, la taille des caches a dû subir une croissance régulière. Toutefois le développement de caches rapides est associé à un coût souvent prohibitif. Pour éviter de faire exploser le prix des processeurs, les architectures modernes fournissent plusieurs niveaux de caches. Sur des processeurs tels que les Core 2 Duo d'Intel, on trouve deux niveaux de cache. Comme le montre la table 2.1, le cache de premier niveau [1] (L1) est très petit (en général de l'ordre de quelques kilo-octets) mais offre une faible latence d'accès. On trouve ensuite un cache de deuxième niveau (L2) plus grand mais associé à une latence plus importante. Sur des processeurs comme le nouveau *Nehalem* d'Intel, on trouve également un troisième niveau de cache commun aux quatre cœurs.

Type de mémoire	Temps d'accès	Taille	Ligne de cache	Voies
Cache L1	4 cycles	2 * 32 Ko	64 o	8
Cache L2	14 cycles	2 - 6 Mo	64 o	8 - 24
RAM	$\sim$ 200 cycles	$\sim$ 1 Go/cœur		

TABLE 2.1.: *Descriptions des caches pour l'architecture Core 2 Duo d'Intel[3]. Attention, les valeurs pour la RAM sont données à titre indicatif, ces dernières sont extrêmement variables en fonction de la configuration matérielle utilisée. Les temps sont donnés en cycles processeurs, correspondants au nombre de tics de l'horloge de ce dernier.*

Lorsqu'une donnée doit être chargée par les unités de traitement du processeur, ce dernier procède comme suit :

1. Recherche dans le cache L1 de données.
2. Si la donnée n'est pas présente dans le cache L1, recherche dans le cache L2 ; si elle est trouvée, la donnée est placée dans le L1 et envoyée aux unités de traitements du processeur.

1. Beaucoup de processeurs disposent de deux caches L1 ; un premier pour les données, l'autre pour les instructions.

3. Si la donnée n'est pas présente dans le cache L2 ; lecture de la donnée dans la mémoire centrale. La donnée est alors placée dans le L2 puis le L1 et envoyée aux unités de traitements du processeur.

On a donc une forme d'inclusion des niveaux de cache pour l'accès aux données. La figure 2.1 montre les interactions des deux cœurs d'un Core 2 Duo d'Intel avec les deux niveaux de cache. Sur ce type de processeurs, le cache L2 est commun aux deux cœurs. Lorsqu'une donnée n'est pas trouvée dans un des caches, on dira par la suite qu'il y a *défaut de cache*.

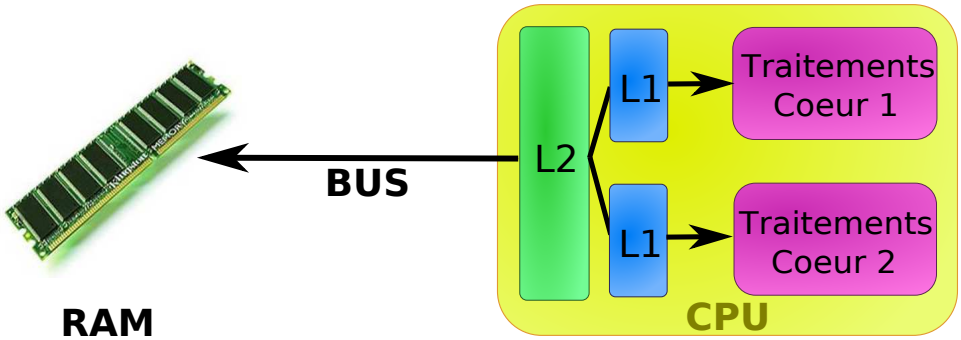

FIGURE 2.1.: *Schéma d'échange de données sur les architectures du type Core2 Duo d'Intel.*

2.1.2. Découpage en lignes/voies

En général les caches ne fonctionnent pas par mots, mais par *lignes de cache* d'une taille de 64 octets. L'utilisation de lignes de cache plus grandes qu'un mot permet de mettre d'un seul coup plusieurs mots dans le cache, en supposant que ces derniers seront utilisés dans un futur proche.

Placer des données dans les caches n'est pas suffisant ; il faut en effet être capable de les retrouver lorsqu'elles sont demandées. Pour cela le processeur doit être capable de déterminer la ligne de cache dans laquelle les données associées à une adresse ont été placées. On dispose de trois techniques pour réaliser cette tâche.

Pleinement associatif : Dans cette approche, les lignes de cache peuvent être associées à n'importe quelle adresse, cela suppose que chaque ligne de cache est associée à un registre mémorisant l'adresse des données qu'elle contient. Lorsqu'une donnée est demandée il faut effectuer une recherche exhaustive sur tout le cache pour trouver la ligne correspondante. Cette technique n'est donc utilisée que sur les très petits caches (quelques kilo-octets).

Association directe : Chaque adresse est associée à une ligne de cache unique, la recherche de données est donc très rapide. L'association est en générale déterminée par la partie basse de l'adresse. Ce mode a toutefois le défaut de créer beaucoup de collisions de lignes de cache si les données utilisées sont associées à la même ligne. Ce mode correspond grossièrement au mode associatif à 1 voie.

Associatif à N voies : Il s'agit d'un compromis entre les deux approches précédentes. Les collisions sont réduites en associant chaque adresse à N lignes de cache. Pour trouver une donnée, on effectue une recherche exhaustive sur au plus N éléments. Les caches processeurs utilisent généralement cette approche pour les caches de tailles importantes. La figure 2.2 représente l'association des segments mémoire avec les lignes de cache de cette approche.

La table 2.1 donne le nombre de voies utilisées pour les différents caches des processeurs de la gamme Core 2 Duo d'Intel.

2.2. Gestion de la mémoire dans un système d'exploitation

Disposer de mémoire n'est pas suffisant, il faut savoir répartir cette précieuse ressource entre les différents processus en satisfaisant des contraintes de fiabilité, rapidité et sécurité. La gestion du

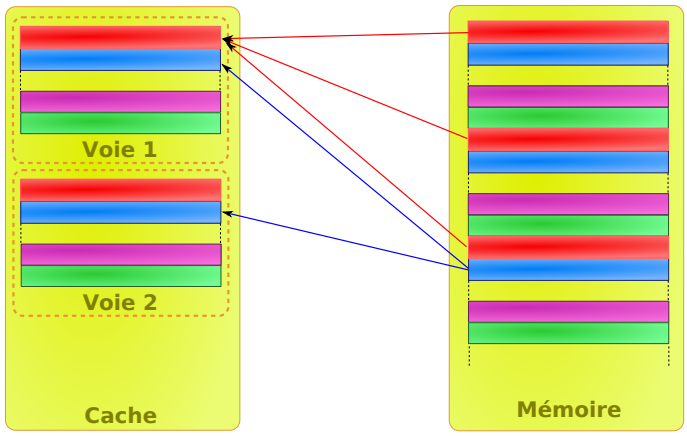

FIGURE 2.2.: *Association des segments mémoire avec les lignes d'un cache associatif à 2 voies.*

partage de la ressource mémoire est l'une des tâches centrales des systèmes d'exploitation. Nous allons voir ici succinctement comment les systèmes d'exploitation (OS) d'aujourd'hui abordent la question. La description qui est donnée ici est minimaliste et n'a pour but que d'éclairer le lecteur quant aux concepts qui seront utiles lors de l'implémentation de notre approche. On pourra trouver plus de détails sur la gestion mémoire des systèmes d'exploitation dans les références [4] et [5].

2.2.1. Adresses virtuelles, adresses physiques

Les architectures modernes disposent d'au moins deux types d'adressage au niveau matériel : *l'adressage physique* dans lequel les adresses correspondent directement à la position des données dans la mémoire centrale, et *l'adressage virtuel* dont les adresses n'ont pas de correspondance directe avec la mémoire centrale. L'utilisation d'un système d'adresses virtuelles offre plusieurs avantages du point de vue de l'OS et des applications.

- Disposer d'un espace d'adressage couvant toute la plage d'adresses accessibles pour une taille d'adresse donnée. Ceci offre un espace d'adressage beaucoup plus grand que l'espace physique qui est restreint à la taille de la mémoire disponible. Disposer de ce grand espace permet de gérer plus simplement et plus efficacement le placement des différents objets dans la mémoire.
- Chaque processus peut disposer de son propre espace d'adressage virtuel ; on peut donc assurer qu'un processus ne pourra pas accéder aux données d'un autre processus de manière non autorisée. Chaque processus est ainsi isolé du reste du système, ce qui offre un bon moyen de sécuriser les données de ces dernières et d'isoler le système d'exploitation.
- Régler la question des codes non relocalisables. Si on ne dispose pas d'un espace virtuel, il faut trouver un moyen de faire cohabiter tous les processus dans la mémoire du système. Or certains programmes utilisent des adresses fixes, il devient alors impossible de faire fonctionner simultanément deux processus de ce type. Les adresses virtuelles sont une manière de régler la question sans être forcé de créer du code relocalisable.
- Mettre en place facilement un système de pagination disque.[2] Dans la suite nous ferons référence à la pagination disque sous Linux par le nom *swap*.

Les architectures de types *x86* utilisent un troisième type d'adresses : *les adresses logiques*, qui avaient été introduites sous DOS pour régler le problème de relocalisation. La technique consiste à associer un offset à chaque segment mémoire décrit par l'OS. Cet offset est utilisé pour décaler les données en mémoire de manière transparente pour l'application. Bien que les OS actuels tirent toujours partie de ce système d'adressage, nous ne le traitons pas ici puisqu'il n'apporte rien quant à nos besoins pour notre approche. Par ailleurs Linux utilise très peu ce système qui est redondant avec la pagination.

Ces différents types d'adresse correspondent aujourd'hui à une implémentation matérielle sur la plupart des processeurs, on peut résumer le flux global de conversion des adresses par la figure 2.3.

2. Par ce biais, il est possible de déplacer les données peu utilisées depuis la mémoire vers un disque dur.

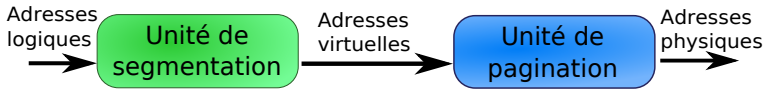

FIGURE 2.3.: *Gestion des conversions d'adresses logiques en adresses matérielles[5].*

2.2.2. La pagination

Pour pouvoir utiliser un système d'adresses virtuelles, il faut être capable de traduire une adresse virtuelle en son équivalent physique. Ce mécanisme est assuré par la pagination, correspondant à une coopération entre le processeur (unité de pagination sur la figure 2.3) et l'OS. Dans cette approche, les espaces d'adressage physiques et virtuels sont découpés en portions dénommées *pages*, occupant 4 Ko par défaut sur *x86*. L'OS doit alors maintenir une table d'adressage décrivant la correspondance de chaque page virtuelle utilisée avec une page physique. L'unité de pagination du processeur utilise alors cette table pour traduire les adresses utilisées par les processus (figure 2.4). Afin d'accélérer cette traduction, le processeur dispose d'un cache local (*Translation Lookaside Buffer* : TLB), permettant de stocker quelques entrées lui évitant d'accéder trop souvent à la mémoire centrale.

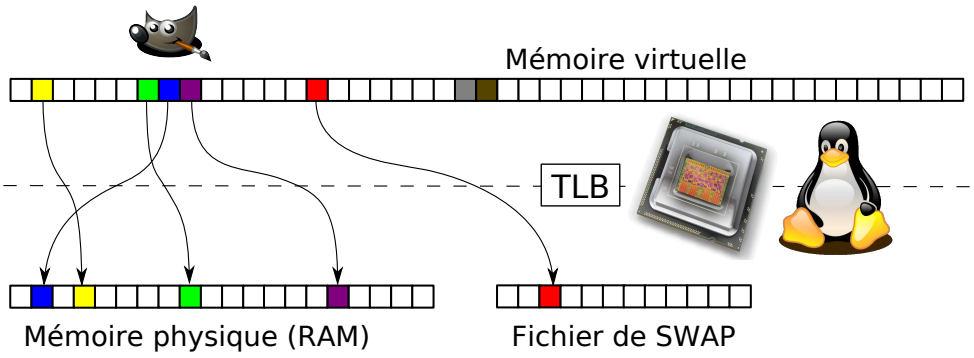

FIGURE 2.4.: *Pagination de la mémoire pour gérer la traduction des adresses virtuelles.*

2.2.3. Défaut de page

Les pages physiques ne sont associées aux pages virtuelles qu'à partir de la première utilisation, et non pas à l'allocation. Un programme peut donc avoir demandé beaucoup de mémoire, mais s'il n'en utilise qu'une petite portion, seule cette portion sera réellement réservée en mémoire physique. On réduit ainsi l'empreinte des processus en mémoire physique. On notera pour la suite que certains programmes prennent ce critère comme postulat et réservent plus de mémoire que nécessaire. Dans notre implémentation, il faudra donc essayer de respecter ce mode de fonctionnement.

Pour satisfaire ce fonctionnement l'OS utilise la méthode du *défaut de page*. Lorsqu'un processus tente d'utiliser une adresse qui n'est associée à aucune page physique, le processeur lève une exception et donne la main au gestionnaire d'exception de l'OS. Ce dernier va alors prendre la faute en charge, déterminer le processus fautif et vérifier qu'il dispose des bons droits d'accès à cette zone mémoire. S'il est autorisé, alors l'OS fournit une page physique et rend la main au processus. Dans le cas contraire, le processus est tué et l'utilisateur obtient une erreur de segmentation.

La faute peut être associée à la violation d'un type de droit d'accès : lecture, écriture ou exécution. L'OS dispose donc d'un moyen pour assurer la protection des données. La faute d'écriture sert également à déterminer si une page utilisée comme cache disque a été modifiée. Si la page a été modifiée, alors cette dernière devra être écrite sur le disque dur avant d'être libérée. Pour cela l'OS n'active la page qu'en lecture. Lorsqu'un processus modifie son contenu, l'OS est notifié par une faute d'écriture, marque la page comme *modifiée* et autorise l'écriture. Un mécanisme similaire est

utilisé pour mettre en place la copie sur écriture lors d'un *fork*, nous reviendrons plus précisément sur ce point dans le chapitre suivant.

2.3. Interaction de la pagination avec les caches

Les modes associatifs des caches présentés dans la section 2.1.2 posent un problème de *collision de lignes de cache*. En effet dans ce fonctionnement, chaque adresse est associée à une ou plusieurs lignes de cache. Si à un instant donné le processus courant essaie d'accéder à des informations associées aux mêmes lignes de cache ; alors le processeur passera son temps à vider et remplir ses lignes de cache ne profitant pas de la taille complète du cache. On rappelle qu'à cause de la pagination, des données contiguës en adresses virtuelles ne le sont pas en mémoire physique comme cela est illustré par la figure 2.5.

Il est donc du ressort de l'OS de mettre en place une politique de gestion de la mémoire qui réduit ces collisions. Cette politique doit se mettre en place au niveau du système de pagination. Alors que FreeBSD et OpenSolaris implémentent une prise en charge de ces collisions, Linux ignore purement et simplement ce problème.

Sous Linux les pages sont en effet distribuées "aléatoirement" aux applications. Il est donc probable qu'une application obtienne successivement deux pages physiques dont l'alignement correspond aux mêmes emplacements dans les voies du cache. Si l'application reçoit plus de pages de ce type qu'il n'y a de voies dans le cache, et qu'elle accèdent régulièrement à l'ensemble de ces données, il y aura collision, et l'application sera ralentie même si toute sa mémoire pourrait rentrer dans le cache. Ce cas est illustré par la figure 2.5.

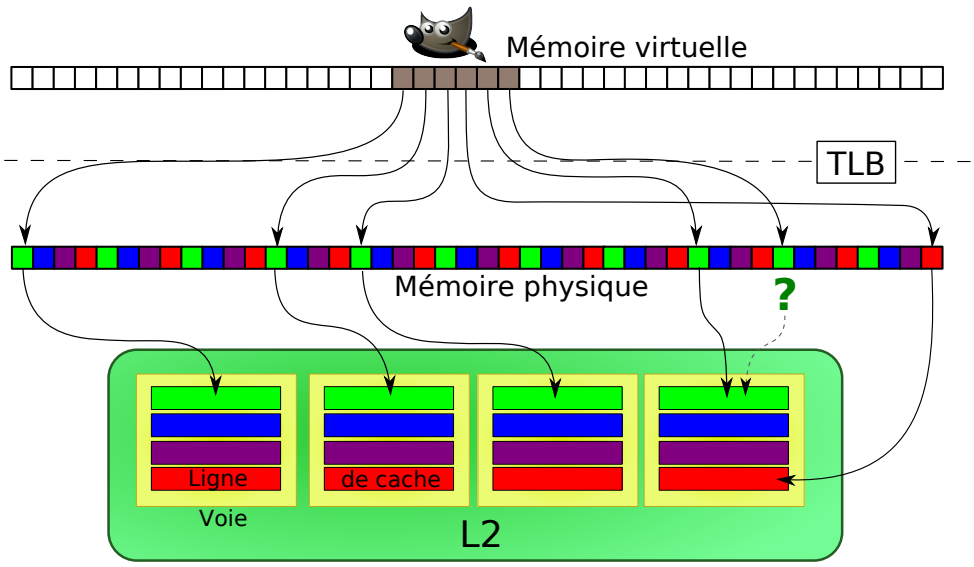

FIGURE 2.5.: *Exemple de conflits de ligne de cache d'une application en supposant un cache L2 disposant de 4 voies. Ici même si toutes les données de The Gimp rentrent dans le cache il y a conflit à cause de la répartition en mémoire des données et les performances seront réduites.*

2.4. Mise en évidence des caches

On peut mettre en évidence la présence des caches en mesurant le temps d'exécution du pseudo code de la figure 2.6 pour différentes tailles de buffer. On notera que pour notre mesure nous avons pré-chargé les caches pour éviter de compter le temps imputé à la première lecture des données en mémoire.

```
ENTIER taille = ....;
DOUBLE buffer[taille];
ENTIER i,j;

PRE_CHAUFFE_DU_CACHE(buffer,taille);

LANCER_CHRONO();
POUR j=0 A 640000000/taille PAR PAS DE 1
    POUR i=0 A taille PAR PAS DE 16
        LIRE(buffer[i]);
    FINPOUR
FINPOUR
ARRETER_CHRONO();
```

FIGURE 2.6.: *Pseudo code de test des caches. PRE_CHAUFFE_DU_CACHE() effectue simplement quelques boucles de lectures du buffer suivant le même type d'accès pour placer les données dans le cache avant le début de la mesure.*

En appliquant cette méthode de mesure sous Linux sur un Xeon Core 2 Quad [3] d'Intel avec de 6 Mo de cache L2, on obtient les temps d'exécutions représentés dans la figure 2.7. On observe clairement l'effet du cache L2 jusqu'à 4 Mo, mais on observe une augmentation du temps d'exécution bien avant les 6 Mo attendus pour notre cache. On confirmera dans la section suivante que cet écart est purement lié à la manière dont les pages sont réparties sous Linux en créant des conflits de lignes de cache beaucoup plus rapidement, comme cela est présenté dans la section 2.3.

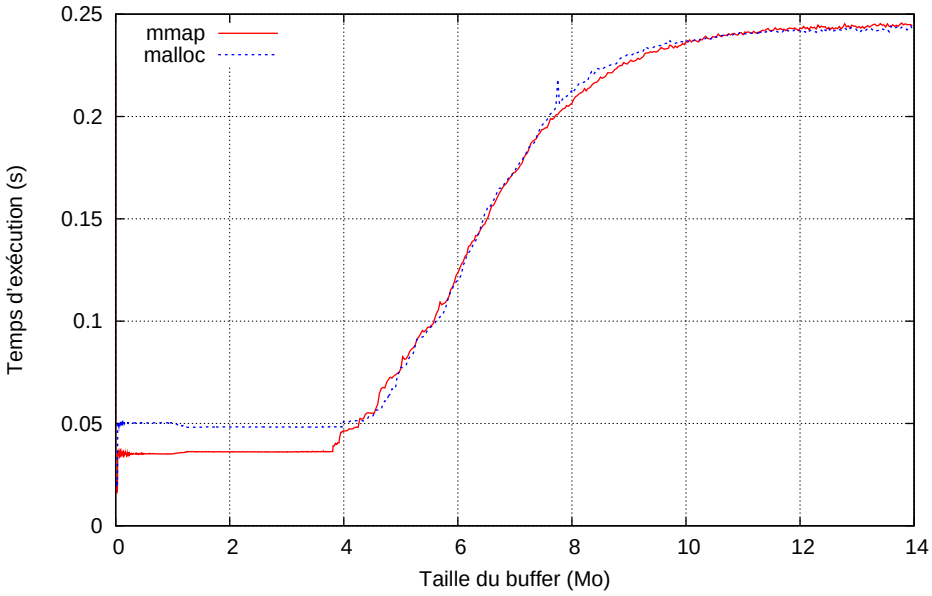

FIGURE 2.7.: *Temps d'exécution du code de la figure 2.6 en fonction de la taille du buffer. Le test a été réalisé sur un Xeon Core 2 Quad d'Intel disposant de 6 Mo de cache L2, en allouant la mémoire avec malloc ou mmap.*

L'utilisation de *malloc* à la place de *mmap* fournit le même résultat avec un palier plus élevé en temps sur le début du cache L2. Ceci est lié au fait que *malloc* renvoie des données alignées sur 16 octets, alors que *mmap* fournit des données alignées sur 64 octets correspondant à la taille des lignes de cache.

3. Référence exacte : Intel Core 2 Q9550.

2.5. Les huge pages sous Linux

2.5.1. Principe et intégration dans Linux

Sous *x86* Linux utilise des pages de 4 Ko, choix qui est contraint par le processeur utilisé, puisque c'est à ce dernier qu'incombe le travail de conversion des adresses. Le tableau 2.2 fournit une liste des tailles supportées par différentes architectures.

Architecture	Tailles de pages supportées
x86 sans PAE	4 Ko et 4 Mo
x86 avec PAE	4 Ko et 2 Mo
x86_64	4 Ko, 2 Mo et 4 Mo
IA64	4 Ko, 8 Ko, 64 Ko, 256 Ko, 1 Mo, 4 Mo, 16 Mo et 256 Mo

TABLE 2.2.: *Liste des tailles supportées par différentes architectures [6]. Sous x86 les choix possibles dépendent de l'activation du mode PAE (Physical Address Extension) permettant d'adresser plus que 4 Go sur un système 32 bits.*

Toutes les architectures listées ici offrent la possibilité de travailler avec des pages de tailles plus importantes. Sur la plupart des processeurs actuels, le TLB ne peut contenir qu'un nombre réduit d'entrées de la table des pages, en général 256, ce qui correspond à un adressage de seulement 1 Mo, soit bien moins que la taille des caches actuels.

Les pages de grosses tailles ont donc été introduites pour résoudre un problème de conflit des TLB. Puisqu'il est très complexe d'augmenter le TLB en gardant de bonnes performances, l'utilisation de pages plus grosses permet d'adresser plus de mémoire avec moins d'entrées.

Il est bien évidemment impossible d'utiliser des pages aussi grosses que 2 Mo pour l'ensemble du système ; une telle utilisation engendrerait un gâchis important de mémoire pour les petites structures, notamment celles du noyau ou de la plupart des applications système. Linux permet donc d'utiliser ce type de pages via un correctif intégré dans la branche officielle du noyau depuis la version 2.6.18. Linux nomme ce type de pages les *huge pages*.

L'utilisation de ces grosses pages n'est pas native pour toutes les applications, ces dernières doivent explicitement demander ce type de page en passant par l'appel système *shmget*[4]. On peut aussi utiliser un appel à *mmap*[5] appliqué sur un système de fichier prévu pour cet effet : *HugeFS*.

2.5.2. Impact sur les caches

Si les pages couvrent entièrement une ou plusieurs voies du cache, on ne peut plus obtenir plus de pages que de voies associées à une position dans les voies. Sur Core 2 Quad, en utilisant des pages de 2 Mo pour un cache de 6 Mo, on remplit le cache avec seulement trois pages, chacune recouvrant entièrement 8 des 24 voies du cache. On attend donc une utilisation optimale du cache. La figure 2.8 montre les temps d'exécutions du code de la figure 2.6 en allouant la mémoire avec les *huge pages* de Linux.

L'utilisation des *huge pages* de Linux nous permet effectivement d'utiliser les caches dans leur totalité. On confirme donc bien que le problème identifié dans la section 2.4 provient du système de pagination de Linux.

On observe un léger renflement entre 1 et 4 Mo sur la courbe de *mmap* par rapport aux *huge pages*. Ce dernier est lié à la faible taille des TLB qui ne peuvent adresser qu'un méga-octet de mémoire.

2.5.3. Utilisation en production

Au vu des résultats de la section précédente, on serait tenté de dire que les *huge pages* sont une solution à notre problème et de nous tourner vers elles pour une utilisation en production sur les

4. L'appel système POSIX *shmget* est utilisé pour allouer des segments de mémoire partagée. On notera que l'utilisation de l'option *SHM_HUGETLB* de cette commande n'est pas conforme à la norme POSIX mais spécifique à Linux.

5. L'appel system POSIX *mmap* permet d'établir des associations mémoire vers des fichiers ou périphériques.

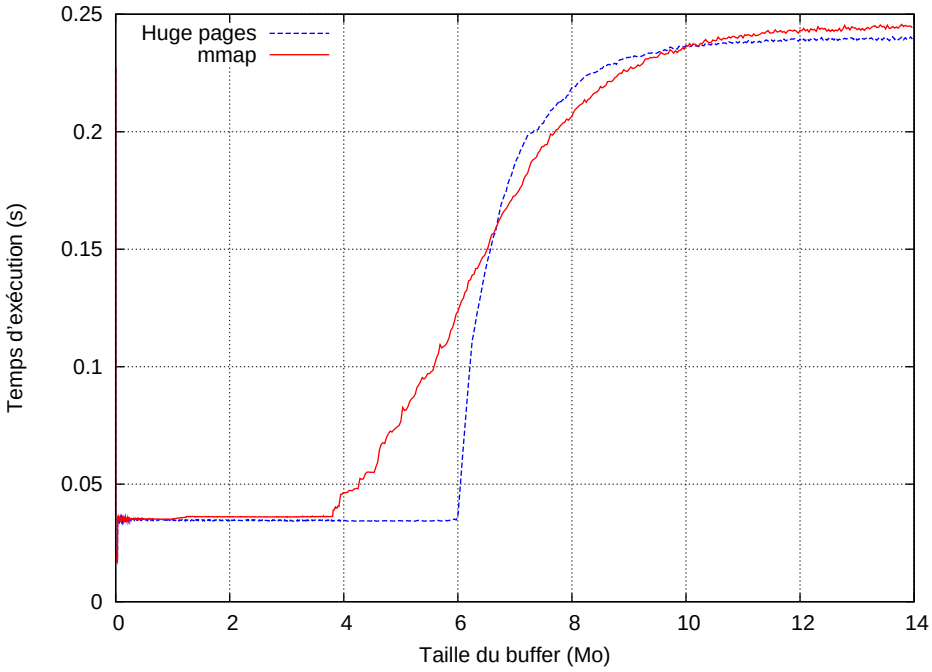

FIGURE 2.8.: *Temps d'exécution du code 2.6 en fonction de la taille du buffer en utilisant mmap avec ou sans les Huge pages. Ce test a été réalisé sur Intel Xeon Core 2 Quad avec de 6 Mo de cache L2.*

calculateurs. Mais leur implémentation actuelle est assez limitée. Ce système est en effet orienté vers les bases de données [7], tout en essayant de limiter au maximum l'impact de la modification sur le sous-système de gestion mémoire de Linux. On trouve donc une série de limitations, qui, bien que moins contraignantes pour les bases de données, sont un problème pour un serveur HPC. Nous allons voir ici différents points qui caractérisent l'utilisation des *huge pages* de Linux.

Réservation de la mémoire : Pour fonctionner, les *huge pages* utilisent une réserve de mémoire contiguë qu'il faut paramétrer. La configuration de ce paramètre n'est en général accessible qu'à l'administrateur. Si l'hôte n'héberge qu'un unique gros serveur de bases de données, le choix est simple : l'administrateur peut allouer toute la mémoire pour cet usage. Dans le cas du HPC il en est autrement. Il est en effet impossible pour l'administrateur de prédire quel sera le fonctionnement des applications qui vont être envoyées sur ses nœuds de calcul. Pré-réserver de la mémoire pour un usage particulier est donc exclu.

Réservation de mémoire contiguë : Pour fonctionner, les *huge pages* doivent arriver à réserver des blocs de mémoire physiquement contigus de la taille des pages à utiliser : 2 Mo ou 4 Mo sur *x86*. En pratique, sur un système fonctionnant depuis plusieurs heures, il est pratiquement impossible d'obtenir un grand nombre de blocs contigus de cette taille. Autoriser l'utilisateur à changer la taille de la réserve n'est donc pas une solution dans notre cas. Pour les serveurs de bases de données, la taille étant connue à l'avance, la réservation est souvent faite au démarrage lorsque la mémoire n'a pas encore été trop sollicitée.

Problème de fuite : Dans certains cas d'utilisation, l'OS peut aller récupérer des pages dans la réserve des *huge pages* pour satisfaire ses propres besoins. Sur un nœud HPC, il y a de fortes chances qu'une réserve même bien choisie aurait tendance à se réduire avec l'utilisation, ce qui n'est pas acceptable.

Arrêt brutal d'application : En cas d'arrêt brutal d'une application utilisant les *huge pages*, la mémoire utilisée par cette dernière risque de rester réservée suivant le mode d'allocation retenu par cette dernière : ceci est inhérent à l'utilisation des mémoires partagées pour accéder à ce type de page.

En résumé, bien que les *huge pages* soient une solution pour notre problème et puissent être utilisées en production pour un serveur de bases de données, il est difficile d'envisager son utilisation pour du HPC. On ajoutera qu'en cas d'allocation de mémoire trop importante par rapport à la réserve, l'application aboutira sur un échec d'allocation. Ce problème peut être contourné en utilisant la bibliothèque *libhugetlbfs*[6] qui utilise des pages standards lorsqu'il n'y a plus de grosses pages disponibles. Mais cette dernière est instable et a tendance à fournir des pages de tailles standards même s'il reste des grosses pages disponibles.

2.6. Pagination dans les autres OS

Nous avons vu dans ce qui précède que Linux ne portait guère attention à sa manière de gérer la pagination vis-à-vis des caches processeurs, nous allons voir dans cette section que FreeBSD et OpenSolaris mettent en œuvre des techniques particulières pour maîtriser ce problème.

2.6.1. Les huge pages / superpages

Nous avons vu dans la section 2.5.2 que l'utilisation de pages plus grosses permettait de réduire voir éliminer les conflits de caches dans le cas de données contiguës en mémoire virtuelle. FreeBSD[8] et OpenSolaris[9] proposent tous deux un système de grosses pages utilisable de manière plus ou moins souple. Dans les deux cas il n'y a pas de réservation à faire, l'OS se charge de récupérer les pages dont il a besoin lorsque c'est nécessaire. On rappelle que Linux manipulait ce type de pages plus ou moins à l'extérieur du sous-système de gestion de la mémoire. Au contraire, OpenSolaris et FreeBSD intègrent ce support de manière native au cœur du système.

Sous OpenSolaris, le choix de taille de page se fait à la compilation du programme ou par l'environnement via une variable informant que l'OS doit utiliser des pages de tailles plus importantes.

Dans la nouvelle version officielle de FreeBSD (7.2) on trouve désormais une implémentation beaucoup plus transparente des grosses pages : les *superpages*. En fournissant un système autoadaptatif, permettant de donner des pages de plus en plus grosses à une application via un système de promotion. Cette implémentation proposée à l'OSDI en 2002[8] s'avère être une manière propre et efficace d'implémenter un tel support. L'intérêt de cette approche est une gestion des grosses pages au cœur du système et indépendante des applications. Le développeur n'a donc pas à se soucier de ces aspects qui sont automatiquement pris en charge par l'OS.

Des implémentations des *superpages* ont été tentées sous Linux IA64/x86_64, on trouve des correctifs pour les versions 2.6.17 et 2.6.24, mais ces derniers sont en version 1 et peu testés[10][11]. On remarquera que Linus Torvalds reste pour l'instant opposé à l'utilisation de ce type de technique dans la branche officielle du noyau.

2.6.2. Coloration de pages

Nous avons vu que l'utilisation de pages plus grosses permettait d'éliminer le problème de collision. En se penchant plus près sur le fonctionnement des caches décrit en section 2.1.2, on peut mettre en évidence le critère important permettant d'éviter les collisions. Ces dernières se produisent si N+1 informations nécessaires à un instant donné correspondent aux mêmes lignes de cache ; N étant le nombre de voies du cache. Les données associées aux mêmes positions dans les voies du cache seront dites de la même couleur.

La condition de contiguïté apportée par les grosses pages élimine effectivement ces cas puisque les données de la page couvrent au minimum une voie complète du cache. Les informations d'une grosse page sont donc toutes distribuées sur des couleurs différentes. Une solution possible est donc d'allouer les pages de telle sorte qu'elles ne correspondent pas aux mêmes couleurs, plutôt qu'utiliser une distribution "aléatoire". Cette technique nommée coloration de page est présente sous FreeBSD et OpenSolaris, ce qui permet à ces OS de mieux utiliser les caches.

Dans cette méthode, chaque page physique est associée à une couleur (le numéro de la ligne de cache à laquelle correspond son adresse). On pourra se référer à la figure 2.5 pour mieux visualiser l'approche. On doit attribuer aux processus des pages de couleurs aussi variées que

6. La bibliothèque *libugetlbfs* fournie une implémentation de *malloc* au dessus des *huge pages* de Linux.

possible. OpenSolaris et FreeBSD maintiennent leur liste de pages libres sous la forme de N listes de couleur et attribuent les pages à la demande selon un choix de règles offertes à l'utilisateur [9] :

Hachage : L'OS détermine la couleur de l'adresse virtuelle en utilisant une méthode de "hachage" et retourne une page de cette couleur. Il s'agit du mode par défaut sur les deux OS.

Tourniquet : La dernière couleur fournie au processus est mémorisée et incrémentée à chaque demande.

Choix statistique : L'OS procède de manière statistique et fournit une page aléatoirement parmi les couleurs les moins renvoyées à l'application.

L'utilisation de la coloration de pages permet de résoudre le problème de conflits de lignes de cache sans ajouter la complexité associée à l'utilisation de pages plus grosses, nécessitant de maintenir une mémoire peu fragmentée et de gérer la cohabitation de pages de tailles différentes. La coloration de page ne règle toutefois pas le problème lié à la taille des TLB.

2.6.3. Résultats sur un exemple simple

En appliquant le code de la figure 2.6 sur les différents OS on obtient les temps d'exécutions représentés sur la figure 2.9. On observe clairement l'efficacité des méthodes utilisées sur FreeBSD et OpenSolaris en ce qui concerne les caches.

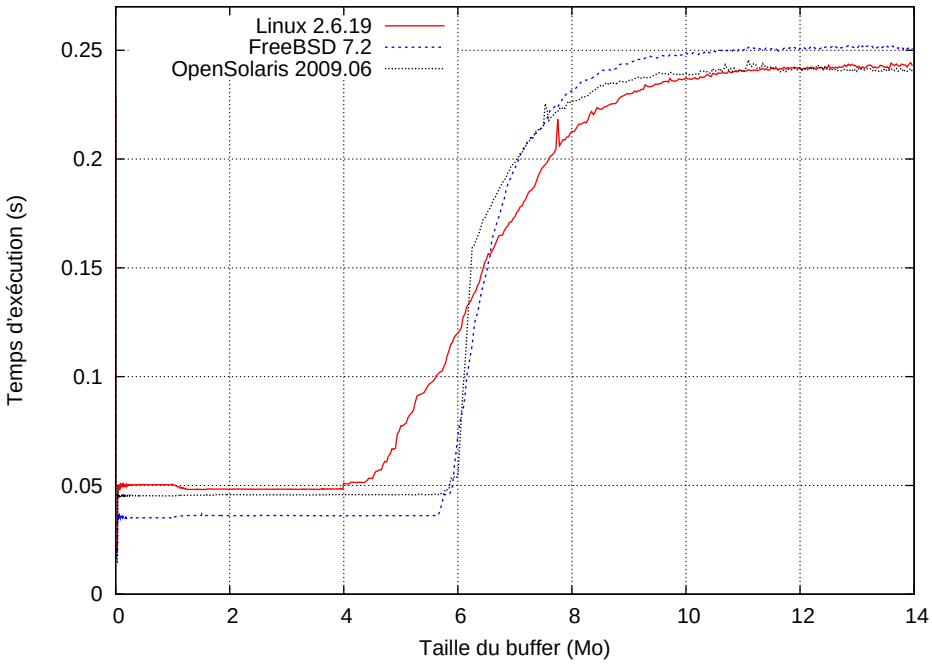

FIGURE 2.9.: *Temps d'exécution du code de la figure 2.6 sur différents systèmes d'exploitation avec malloc pour différentes tailles de buffer.*

3. Notre solution : Virtual Huge Pages

Maintenant que nous avons fait un rappel sur la gestion de la mémoire dans les systèmes d'exploitation, nous allons pouvoir décrire l'approche proposée pendant le stage pour régler le problème de fuite des caches observée sous Linux dans la section 2.4. Nous allons dans un premier temps décrire les différents critères qui seront nécessaires aux choix de notre implémentation. Puis nous décrirons la manière dont l'espace utilisateur devra interagir avec l'espace noyau. Enfin nous décrirons en détail quelques points particuliers de notre implémentation.

3.1. Objectifs et contraintes

Au cours de ce stage nous avons cherché à mettre en place une solution permettant de compenser le problème de fuite de cache sous Linux (voir section 2.4) dans le cadre des applications HPC du CEA/DAM. Nous allons voir ici les critères que nous allons devoir prendre en compte dans notre solution.

Réduire les collisions de lignes de cache : Il s'agit de notre objectif principal. Nous avons vu dans la section 2.5.2 et 2.6.2 que deux solutions permettent de régler ce problème : l'utilisation de pages plus grosses ou la coloration de pages. L'objectif du stage est de tenter une approche par l'utilisation de grosses pages.

Éviter les problèmes des huge pages : Nous avons listé un certain nombre de problèmes liés aux *huge pages* de Linux pour le HPC (section 2.5), notre solution devra bien évidemment éviter d'imposer les mêmes contraintes pour être viable.

Thread-safe : Avec l'utilisation de processeurs multi-cœurs et machines multiprocesseurs, il est bien évident que notre solution devra supporter le multi-thread et si possible ne pas trop dégrader les performances par ses mécanismes de protection et synchronisation. On notera que Linux est un OS fonctionnant en mode préemptible, [1] il est donc de toute façon nécessaire de prendre en charge les problèmes de synchronisations même en mode mono-CPU.

Intégration dans les applications : Bien que le CEA/DAM implémente ses propres applications, il serait préférable que notre approche puisse être utilisée par ces dernières le plus simplement possible, en évitant de modifier leur code source.

Correctif noyau : Dans le cadre des applications HPC du CEA il est difficilement envisageable, pour des raisons de sécurité, d'utiliser un correctif noyau sur les nœuds de calcul.

Correction dans le noyau officiel : Un correctif noyau pourrait être utilisé au CEA s'il était intégré dans la branche officielle. Toutefois faire accepter un tel correctif dans le sous-système mémoire n'est pas envisageable sans compter une longue période de discussions et de tests. On notera que Linus Torvalds reste encore aujourd'hui opposé à l'introduction de systèmes de type *superpages* dans le cœur de son noyau ; ce type de solution n'apportant pas de bénéfice pour tous les types d'utilisations.

Utilisation d'un module : il s'agit de la seule approche envisageable puisqu'il n'y a pas besoin de modifier le noyau. L'interface noyau offerte aux modules variant moins que l'intérieur du noyau il y a moins de chances d'incompatibilités entre versions proches. Ce choix offre également une approche plus souple pour le développement et la maintenance de la solution.

Temps d'allocation : Il est bien évident que notre solution ne doit pas dégrader de manière trop importante le temps d'allocation mémoire.

1. Un système est dit préemptible s'il peut à tout moment être interrompu pendant ses traitements noyau, par un évènement externe, basculer sur le gestionnaire de cet évènement pour enfin revenir sur son flux normal d'exécution. La préemption est une source de concurrence, même en mode mono-CPU.

Large utilisation de la mémoire : Nous sommes dans le cadre du HPC, les applications ont donc une certaine tendance à consommer beaucoup de mémoire, on pourra prendre ce critère en compte pour diriger nos choix d'optimisations.

3.2. Description générale de la solution retenue

Nous avons opté pour une implémentation d'une solution sous forme d'un module noyau, l'interface est ainsi clairement définie, moins dépendante des aspects les plus subtils du noyau et plus simple à mettre en place. Développer un module noyau est également plus simple étant donné qu'une modification n'impose pas un redémarrage complet de la machine de test.

Une implémentation en tant que module apporte une contrainte forte sur nos marges de manœuvres. En effet, le noyau n'expose pas toutes ses méthodes aux modules, et ceci est particulièrement vrai pour le sous-système mémoire qui n'est pas conçu pour être ré-implémenté dans un module.

Pour régler le problème de cache, le stage est orienté vers l'utilisation de pages de tailles plus importantes. Pour éviter de devoir réserver de la mémoire à l'avance, notre solution doit pouvoir trouver ses pages dynamiquement. L'implémentation des *huge pages* de Linux n'expose aucune méthode aux modules et il nous est impossible sans réécriture importante de travailler directement sur les tables de pages de l'OS. Notre solution devra donc se baser sur des pages standards de 4 Ko sur *x86*.

Nous avons donc proposé une solution d'utilisation de pages virtuellement plus grosses, construites à partir d'un nombre variable de pages standards contiguës. Par la suite nous utiliserons le terme *page virtuelle* pour désigner ces pages composées. N'étant plus limité par le support matériel de tailles particulières, nous pouvons tenter d'utiliser des pages plus petites que 2 Mo ou 4 Mo, ce qui devrait permettre de réduire la gêne associée à la fragmentation de la mémoire.

Notre solution portera donc le nom de *Virtual Huge Pages*, que l'on abrègera dans la suite par *VHP*. Pour implémenter notre solution, nous nous sommes largement appuyés sur l'aide précieuse fournie par la référence [12]. On pourra aussi se référer aux sites *www.mm-linux.org* et *lwn.net*.

3.3. Interface espace utilisateur / espace noyau

Les architectures modernes offrent au noyau un moyen de protection via des niveaux d'exécution. Bien que la norme *x86* offre 4 niveaux, Linux n'en utilise que 2 : le niveau 0 pour *l'espace noyau* et le niveau 3 pour *l'espace utilisateur*. L'utilisation de ces deux espaces permet d'interdire les instructions dangereuses aux processus utilisateurs. Le noyau est ainsi totalement isolé de ces derniers qui doivent impérativement passer par lui pour effectuer toutes actions nécessitant ce type d'opération : les accès matériels, la modification des TLB, ...

Pour implémenter un module noyau, il faut commencer par définir la manière dont les processus en espace utilisateur vont devoir communiquer avec l'espace noyau. Pour cela, l'OS fournit les *appels systèmes* comme points d'entrées. Lorsqu'un processus fait un appel système, le processeur passe en espace noyau pour effectuer les traitements demandés. Les appels système sont clairement définis et tous les paramètres fournis par l'utilisateur sont conditionnés et vérifiés avant d'aboutir sur les méthodes de traitement du noyau. On assure ainsi l'étanchéité entre les deux espaces.

Notre implémentation est réalisée à l'intérieur d'un module, nous ne pouvons donc pas créer de nouveaux types d'appels systèmes. Dans le cadre de la gestion de la mémoire il est naturel d'utiliser les appels *mmap* et *munmap* que nous détaillerons dans la section 3.3.2. À moins d'utiliser l'option *MAP_ANONYMOUS* les appels à *mmap* et *munmap* doivent être associés à un descripteur de fichier.

3.3.1. Le périphérique

Nous offrons donc un accès via un périphérique de type caractère [2] qui correspond bien à notre approche. Nous fournissons donc un tel périphérique avec les méthodes de manipulation requises

2. Les périphériques caractères permettent d'accéder aux données du périphérique comme s'il s'agissait d'un tableau en mémoire.

par Linux. Dans notre cas, l'implémentation des méthodes *open*, *release* et *mmap* suffiront pour le descripteur.

Sous Linux, chaque périphérique à l'exception des cartes réseaux, est représenté par un fichier dans l'arborescence du dossier */dev*. Notre implémentation ajoutera deux fichiers à ce dossier :

/dev/vhp Périphérique d'allocation de pages virtuelles.

/dev/vhp_color Périphérique d'allocation utilisant de la coloration de pages pour effectuer quelques comparaisons.

Les fichiers du dossier */dev* sont créés par la commande *mknod* en fournissant les identifiants du périphérique. Dans les versions récentes du noyau les identifiants sont associés de manière dynamique, on utilise donc un script capable de les trouver et de créer les fichiers correspondants. À terme il sera nécessaire de remplacer ce script par un fichier de configuration dans *udev*[3] qui est la manière standard de créer les fichiers périphériques.

Pour utiliser notre système d'allocation, l'application devra ouvrir un des deux périphériques et utiliser la méthode *mmap* pour réclamer de la mémoire.

3.3.2. Fonctionnement de mmap / munmap

Toute notre implémentation repose sur l'appel système *mmap*, nous allons donc prendre le temps de décrire son comportement. L'appel *mmap* permet d'associer directement une plage de la mémoire virtuelle d'un processus aux caches d'un fichier ou d'un périphérique afin d'en accélérer les accès. Nous fournissons donc via le périphérique le moyen de remplir cet espace mémoire à l'aide de nos pages virtuelles.

Sous Linux, la mémoire virtuelle d'un processus est découpée en VMA[4]. Une VMA est créée à chaque appel de *mmap* et chaque VMA peut fournir ses propres méthodes de gestion via une structure du type *vm_operations_struct*. Lorsqu'un évènement survient à une adresse donnée (par exemple un défaut de page), Linux détermine la VMA correspondante et invoque les méthodes associées à l'évènement. Nous fournissons donc les méthodes suivantes [12] :

vhp_vm_open est invoquée lorsqu'une VMA doit être découpée ou dupliquée.

vhp_vm_close est invoquée à la fermeture d'une VMA : lors d'un appel à *munmap*, d'une fermeture du fichier ou de l'arrêt brutal de l'application. Grâce à cette méthode nous pouvons contourner l'un des problèmes de l'implémentation des *huge pages*, en évitant de laisser de la mémoire réservée en cas d'arrêt brutal de l'application.

vhp_vm_fault est invoquée lorsqu'une faute de page survient ; nous l'utilisons pour supporter l'allocation au premier accès.

vhp_vm_mkwrite est invoquée lorsqu'un défaut d'écriture survient permettant le support de la copie sur écriture après un appel à *fork*.

3.3.3. Les options de mmap

L'appel système *mmap* propose plusieurs modes de fonctionnement que nous allons rapidement décrire. Pour plus de détails sur ces modes, consultez le manuel de l'appel.

MAP_FIXED : Ce mode est utilisé pour forcer la position de la VMA dans l'espace virtuel de l'application. Sans cette option *mmap* choisit lui-même une adresse.

MAP_PRIVATE : Ce mode correspond au comportement par défaut de la mémoire des processus, en utilisant la copie sur écriture après un *fork*. Avec cette option le noyau prend en charge lui-même les évènements de type faute de page, ou faute d'écriture.

MAP_SHARED : Les VMA allouées avec ce mode sont partagées entre tous les processus de la filiation créée par les appels à *fork*. Dans ce cas les notifications sont toutes renvoyées au module.

3. udev est un démon chargé de créer automatiquement les fichiers correspondants au périphérique lorsque le module associé est chargé.
4. Virtual Memory Area

MAP_ANONYMOUS : Ce mode permet d'utiliser *mmap* pour l'allocation mémoire standard des processus. Par exemple, *malloc* utilise *mmap* avec cette option pour allouer les tableaux de grandes tailles. Dans ce cas la VMA n'est associée à aucun descripteur de fichier. Ce mode n'est donc pas utilisable pour notre implémentation.

Pour notre approche nous devons fournir un support du mode *MAP_PRIVATE* afin ne pas modifier le comportement des applications. Or dans ce mode, nous ne recevons plus les notifications nécessaires à nos traitements. Nous allons donc remplacer l'option par *MAP_SHARED* au début de la méthode *vhp_file_mmap* et émuler nous-mêmes le fonctionnement du mode *MAP_PRIVATE*.

Dans le futur nous pourrons également fournir un réel support du mode *MAP_SHARED*. La figure 3.1 résume les interactions entre les différents sous-systèmes en espace noyau et utilisateur.

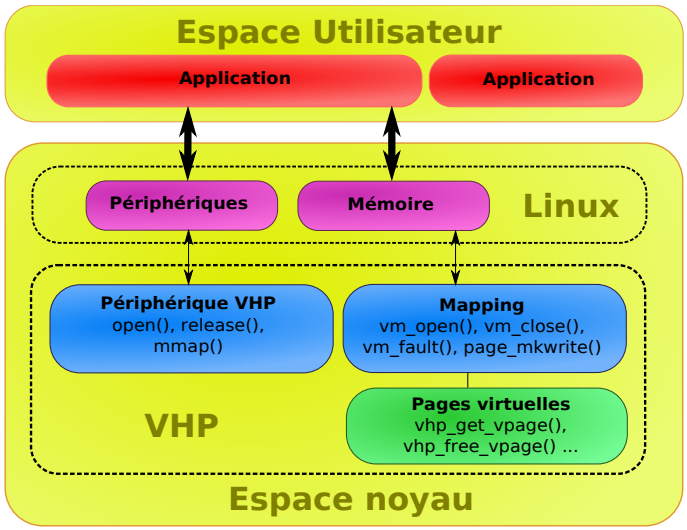

FIGURE 3.1.: *Interactions entre les différents sous-systèmes en espace noyau et utilisateur dans notre implémentation.*

3.4. Structures de données du module

3.4.1. Description générale

Linux maintient une liste des pages contiguës libres sous forme de blocs constitués d'un nombre de pages défini par une puissance de deux. On utilise ce système pour obtenir la mémoire nécessaire à nos pages virtuelles. Pour des tailles importantes nous ne pouvons pas être assurés d'obtenir de telles pages. Nous utilisons donc une méthode dégressive. S'il n'y a plus de blocs contigus de taille suffisante, nous devons utiliser des blocs de tailles plus petites, jusqu'à obtenir dans le pire des cas, des pages standards uniques. Nous devons donc gérer la présence de pages de tailles variables.

Comme nous devons supporter l'allocation au premier accès, nous devons avoir mémorisé la position des zones déjà allouées pour ajuster au mieux la taille et la position des pages dans les zones libres. Nous sauvegardons donc l'état de l'allocation dans une structure dynamique que l'on attache aux VMA à l'aide du champ *vm_private_data* de la structure décrivant les VMA dans le noyau (*vm_area_struct*). Cette structure est maintenue à jour par nos méthodes de gestion des évènements de la VMA.

Nous devons aussi pouvoir partager la description des pages virtuelles entre plusieurs VMA pour pouvoir supporter le *fork* en mode copie sur écriture ou pour éventuellement supporter correctement le mode *MAP_SHARED*. Nous représentons donc l'état de l'allocation sous la forme donnée par la figure 3.2.

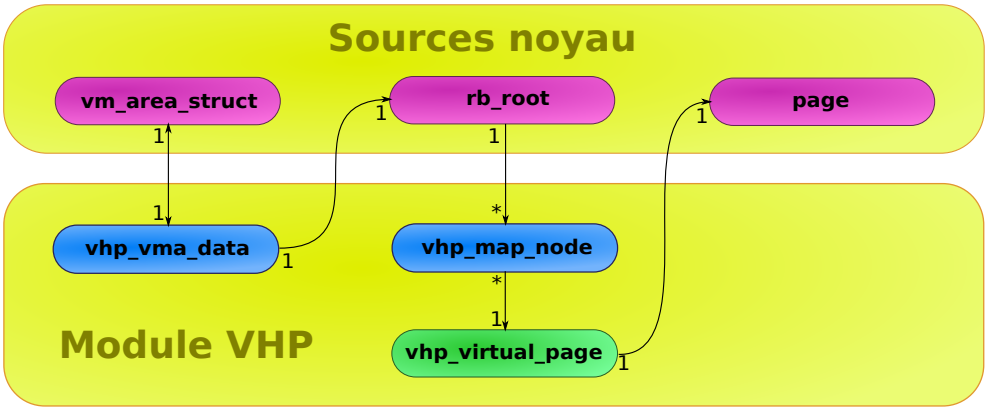

FIGURE 3.2.: *Relations entre les différentes structures importantes utilisées par le module.*

3.4.2. Membres importants des structures

Nous allons voir ici plus en détail les membres importants des structures du module extraites dans la figure 3.3.

```
struct vhp_vma_data
{
    struct rb_root tree;
    struct semaphore mutex;
    struct vm_area_struct * vma;
    pgoff_t gbloff;
};

struct vhp_map_node
{
    struct rb_node node;
    struct vhp_virtual_page * vpage;
    long pgoff;
};

struct vhp_virtual_page
{
    struct page * f_page;
    unsigned int order;
    atomic_t count;
};
```

FIGURE 3.3.: *Extraits des membres importants des structures du module.*

vhp_vma_data est utilisée pour stocker les informations nécessaires au fonctionnement du module. Rattachée directement au niveau de la VMA, elle est allouée, puis libérée en même temps que la VMA par les méthodes *vhp_file_mmap, vhp_vm_open, vhp_vm_close*. Cette structure contient entre autres les informations de l'état actuel de l'allocation et le mutex nécessaire à la protection des informations. L'offset *pgoff* permet de décaler l'origine des positions des pages virtuelles pour accélérer les découpes de VMA lors des appels à *munmap*. Cet offset pourra également être utilisé pour corriger les alignements en mémoire virtuelle si l'on désire intégrer les *huge pages* à notre module.

vhp_map_node permet d'attacher les pages virtuelles à la structure *vhp_vma_data*. On peut ainsi partager les pages virtuelles entre plusieurs VMA. La position de la page dans la VMA est

stockée dans cette structure. Les nœuds sont reliés à *vhp_vma_data* par un arbre binaire symétrique (*Red-Black Tree*) dont l'implémentation est fournie par les sources du noyau. On garantit ainsi un fonctionnement en $O(\log(n))$ pour l'insertion, la recherche et la suppression.

vhp_virtual_page définit une page virtuelle. Il s'agit simplement de l'adresse de la première page du bloc de pages contiguës, associée à un compteur de référence atomique, et de la taille du bloc donné en puissance de deux (*order*). Le compteur permet de savoir si la page peut être libérée ou non, et si elle doit être copiée lors d'un accès en écriture. Les deux autres membres sont utilisés lors des échanges avec les méthodes du noyau.

3.5. Fragmentation de l'allocation

L'une des difficultés de l'approche retenue est la gestion de la fragmentation de la mémoire. En effet, l'utilisation de pages virtuelles de tailles variables nous oblige à devoir faire attention à la manière de distribuer nos pages dans la VMA, ce afin d'utiliser un maximum de pages virtuelles de tailles maximales pour résoudre les conflits de lignes de cache. Maintenir cette distribution est compliqué par l'utilisation de l'allocation au premier accès, puisque nous ne sommes plus maîtres de l'ordre dans lequel les pages sont allouées.

Pour résoudre ce problème nous utilisons la procédure suivante pour allouer une page à une adresse donnée dans la VMA :

1. Calcul des limites droite et gauche de la zone non allouée contenant l'adresse cible. Ceci afin d'éviter les chevauchements.
2. Calcul de la taille maximale permettant de couvrir cet espace. On rappelle que Linux utilise un *buddy allocator*, nous obligeant à utiliser des tailles sous la forme de puissances de deux dont la puissance est 12 au maximum sur x86 (16 Mo). Notre module utilise également un paramètre externe pour limiter la taille des pages virtuelles.
3. Calcul de la position de la page afin de maintenir un alignement correspondant à sa taille et contenant l'adresse cible.
4. Correction de la taille si nécessaire pour ajuster l'alignement requis par les limites de la zone libre.
5. Allocation d'une page virtuelle correspondant à la taille calculée.
6. Si la page obtenue n'a pas la bonne taille (cas d'une mémoire libre très fragmentée), on recalcule l'adresse avec le nouvel alignement de manière à contenir l'adresse cible.

La figure 3.4 montre un exemple d'application de cette méthode. On obtient une couverture optimale de la VMA à l'exception de la fin de cette dernière si elle n'a pas une taille multiple de celle des pages virtuelles utilisées. On notera que cette approche est également utilisée dans l'implémentation des *superpages* de FreeBSD [8]. L'utilisation de l'alignement fonction de la taille des pages permet d'éviter de fragmenter l'espace virtuel si l'application accède à ses données dans un ordre aléatoire. Cette approche peut également nous permettre d'utiliser les *huge pages* dans notre module en appliquant un correctif mineur au noyau pour exporter les méthodes associées. Pour pouvoir être utilisées, les *huge pages* doivent en effet être alignées en espace physique et virtuel.

3.6. Support de l'allocation au premier accès

Supporter ce type d'allocation est un pré-requis pour un certain nombre d'applications qui prennent ce fonctionnement comme acquit et allouent plus de mémoire que nécessaire. Plusieurs complications viennent toutefois entourer l'implémentation de ce comportement. Nous avons vu dans la section précédente comment le problème de fragmentation a été résolu pour supporter cette implémentation, il n'y a donc plus de problème du côté des structures utilisées par VHP. On remarquera également que l'utilisation d'un arbre symétrique permet de garantir un fonctionnement en $O(n \ln(n))$ pour peupler toute la VMA.

L'allocation au premier accès passe obligatoirement par l'interception des fautes de pages pour allouer la mémoire avant de rendre la main au processus. De ce point de vue, Linux offre tout

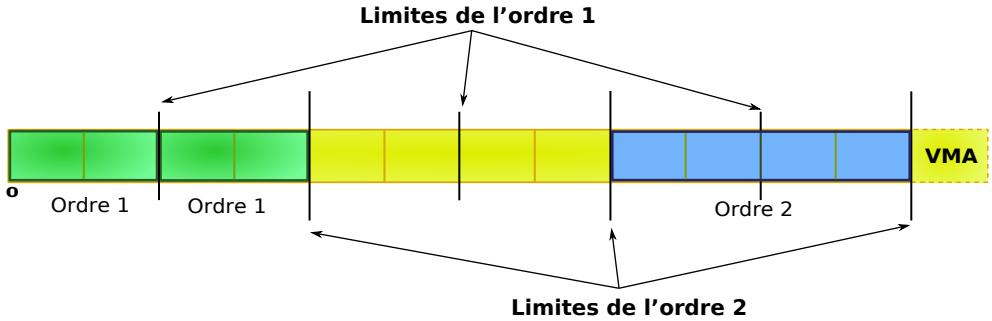

FIGURE 3.4.: *Exemple de gestion de la fragmentation de la VMA. Si en position 0 nous avons obtenu une page d'ordre 1, on complète à la position 2 par une page d'ordre 1 pour satisfaire les alignements.*

le nécessaire puisque les VMA permettent de fournir notre propre implémentation de gestion des fautes de pages. On se trouve toutefois face à deux questions plus ou moins délicates sur ce point.

Comme cela est discuté dans les références [13, 14] le prototype de la méthode a été modifié après la version 2.6.23 du noyau. Dans les versions précédentes le noyau attendait une page en retour de la méthode. Dans ce contexte il est impossible de retourner plusieurs pages, ce qui oblige l'implémentation à retourner séparément chaque page standard de nos pages virtuelles. Par chance, le nouveau prototype est beaucoup plus souple et nous permet de placer nous-mêmes nos pages dans la VMA. On s'en tiendra à cette implémentation qui implique que notre module ne supporte que les versions du noyau supérieures à 2.6.23. Les anciens noyaux peuvent être supportés sans grandes modifications en allouant les pages au moment de l'appel à *mmap*.

Fournir ce type d'allocation expose également le module à un nombre accru d'accès concurrents. Dans le cas d'une application multi-thread, il y a des chances que les threads accèdent simultanément à des zones mémoires encore non associées. Dans un premier temps nous répondons à la question en utilisant un unique mutex sur la structure *vhp_vma_data* mais ce dernier représente clairement un frein au flux normal d'exécution. Il faudra évaluer par la suite si ce choix est acceptable ou critique sur des applications réelles. On pourra éventuellement proposer l'utilisation d'un *spinlock* qui devrait être moins coûteux bien que moins adapté au contexte interruptible dans lequel est exécuté notre code.

3.7. Support des appels à munmap

Les applications utilisant *mmap* peuvent à tout moment annuler les allocations mises en place à l'aide de l'appel système *munmap*. On notera que cet appel n'est pas restreint à s'appliquer à la totalité de la VMA. Le programmeur peut ainsi libérer une petite portion d'une VMA créée avec *mmap*. Dans ce cas le noyau effectue les opérations suivantes pour prendre en charge la découpe :

1. Une nouvelle VMA est créée pour contenir la portion à supprimer.
2. Les pages allouées dans la portion concernée sont déplacées depuis la VMA d'origine vers la nouvelle VMA.
3. La méthode *open()* est appelée sur la nouvelle VMA pour initialiser les structures.
4. La nouvelle VMA est détruite ce qui provoque l'appel de la méthode *vhp_vm_close*.

Nos structures maintiennent une description des pages allouées par les VMA de notre périphérique. On doit donc maintenir cette description synchrone avec le contenu en pages des VMA utilisées. Dans le cas d'un appel à *munmap* on doit effectuer les mêmes opérations que celles réalisées dans le noyau. A savoir créer une nouvelle structure *vhp_vma_data*, y déplacer les pages correspondantes et détruire la structure lors de l'appel à *vhp_vm_close*.

Contrairement au cas du noyau, l'utilisation de pages de tailles non standards nous oblige potentiellement à partager certaines pages entre deux VMA si la coupure se situe au milieu d'une page virtuelle. Après la fermeture, la page virtuelle ne sera alors plus utilisée entièrement. Conduisant à une légère perte de mémoire tant que la VMA voisine n'est pas elle aussi supprimée.

Ce problème pourrait se résoudre en fragmentant la page virtuelle, mais les méthodes nécessaires à cette opération ne sont pas exportées. L'autre solution consiste à allouer une nouvelle page virtuelle plus petite et copier le contenu avant de remplacer l'association. Cette dernière solution risque toutefois de ralentir les opérations. Dans un premier temps nous choisissons la légère perte de mémoire et reviendrons sur ce point si un gâchis trop important est constaté en pratique.

La détection d'un appel de type *munmap* doit être faite au niveau de l'implémentation de la méthode *vhp_vm_open* de notre module. Pour cela, on utilise le champ *vma* de la structure *vhp_vma_data*. Si les adresses de la VMA passées en paramètre ne correspondent pas à celles de la VMA associée à la structure on saura qu'il y a eu appel à *munmap*. Dans le cas contraire il s'agit d'un *fork* que l'on va traiter dans la suite.

3.8. Support des appels à mremap

Linux propose un appel système supplémentaire non standard : *mremap*. Grâce à cet appel nous pouvons modifier la taille d'une VMA afin de la réduire ou de l'agrandir. Cet appel ne provoque toutefois aucune notification de notre module. Notre implémentation supporte naturellement le cas de l'agrandissement grâce à l'allocation au premier accès. Si la VMA a été agrandie, les fautes de pages seront traitées correctement pour s'adapter à la nouvelle taille.

Le cas de la réduction est plus complexe et n'est pas pris en compte dans l'implémentation actuelle, de telle sorte que la mémoire allouée ne sera pas libérée par le module. Ce cas peut être amélioré en partie en évitant de maintenir des références vers les pages dans le module ; ceci au prix d'une complication des méthodes utilisées pour supporter le *fork*. La mise à jour de nos structures pourrait alors être réalisée en testant la taille de la VMA lors des appels à *vhp_vm_fault* afin de détecter un changement de taille. Cette procédure ne prend toutefois pas en compte le cas pathologique d'une réduction puis augmentation de la taille de la VMA sans appel intermédiaire à nos méthodes.

Pour l'instant on considèrera donc la réduction de VMA via *mremap* comme interdite.

3.9. Support des appels à forks

3.9.1. Fonctionnement au fork

L'appel système *fork* permet à un processus de se dupliquer. Le clone peut alors continuer le même flux d'exécution ou en suivre un autre selon les choix du développeur. La plupart du temps un tel appel est suivi de l'appel système *exec* pour charger une nouvelle application.

Lorsqu'un processus est copié de la sorte, toutes ses VMA sont copiées et les méthodes *open* de ces dernières sont appelées. Lors de ces appels nous devons simplement créer une copie de la structure *vhp_vma_data* source. Nous devons toutefois faire attention au comportement fourni. Sous Linux la mémoire d'un processus ayant fait un *fork* est en général partagée entre les deux processus en mode "copie sur écriture". Ceci permet d'économiser l'allocation de nouvelles zones mémoires et la copie des contenus si le processus cloné ne les modifie pas. À la première modification, les pages concernées sont effectivement copiées, afin de maintenir la cohérence des données et l'isolement des deux processus.

Comme nous désirons remplacer le système d'allocation normale des applications nous allons devoir maintenir ce comportement. Lors du *fork* nous allons donc simplement copier les structures *vhp_map_node* et les faire pointer sur les mêmes pages virtuelles que leurs originaux. Les deux VMA utiliseront donc les mêmes pages comme le montre la figure 3.5.

3.9.2. Détection de l'écriture

Pour détecter une écriture nous allons devoir basculer les drapeaux correspondants dans la table des pages de la VMA pour à nouveau recevoir les notifications *page_mkwrite*. La difficulté réside dans l'accès à cette table. Depuis un module nous n'avons en effet accès qu'à un nombre restreint de ses méthodes associées. Plus gênant, toutes les méthodes qui nous permettraient d'effectuer simplement cette opération sont restreintes au noyau.

Pour la VMA clonée, nous pouvons résoudre le problème assez simplement. On remarque en étudiant les méthodes de copie du noyau, que ces dernières ne copient le contenu de la table des pages qu'en présence des options *MAP_SHARED* et *VM_INSERTPAGE* sur la VMA. La première n'est pas une option pour nous si l'on veut recevoir les notifications. La seconde est utilisée à des fins d'optimisation pour ne pas perdre de temps à copier le contenu des VMA au moment du *fork* s'il n'y a pas certains types de pages : entre autres des pages insérées par la méthode *vm_insert_page* que nous utilisons. On remarquera aussi que cette option n'est pas utilisée ailleurs dans le noyau. Nous l'avons donc simplement annulée après chaque appel à *vm_insert_page* pour obtenir une VMA clonée vierge. Nous pouvons alors utiliser *vm_insert_page* pour replacer nous-mêmes les pages qui seront alors insérées avec l'option d'écriture désactivée.

En ce qui concerne la VMA source, il nous faut également parvenir à remettre en place les drapeaux d'écriture. Nous n'avons à priori pas d'autre solution qu'utiliser la méthode *zap_vma_ptes* fournie depuis la version 2.6.27 du noyau. Cette méthode permet de vider un segment de la table des pages d'une VMA. On remarque notamment que l'implémentation de cette méthode vérifie la présence du drapeau *VM_PFNMAP* sur la VMA. Ce drapeau similaire à *VM_INSERTPAGE* est activé lorsque des pages sont placées dans la VMA avec *remap_pfn_range*. Pour pouvoir vider notre VMA nous sommes donc obligés d'activer le drapeau *VM_PFNMAP* puis de le désactiver en fin d'appel. Cette procédure ne pose à priori pas de problème puisque ce drapeau est utilisé comme *VM_INSERTPAGE*. En pratique il sera certainement possible de laisser l'option *VM_PFNMAP* active.

Une fois la VMA vidée de ses pages, nous pouvons alors remettre en place les associations nous-mêmes, comme nous l'avons fait pour le clone. Il est clair que cette procédure est lourde, mais il n'y a à priori aucune autre alternative depuis un module. On notera que les mutex des VMA sont activés lors du *fork*, nous ne risquons donc pas d'aboutir sur un conflit pendant que notre procédure vide puis remet en place les pages dans la VMA source. Le fork n'étant pas une opération très fréquente et déjà très lente, nous pourrons aisément accepter ce surcoût.

3.9.3. Support de la copie sur écriture

Maintenant que le *fork* a été mis en place nous pouvons recevoir correctement les notifications nécessaires à la copie sur écriture. Ce mode est supporté en appliquant le traitement suivant lorsqu'une faute d'écriture est relevée par le noyau et transmise à notre méthode *vhp_vm_mkwrite*.

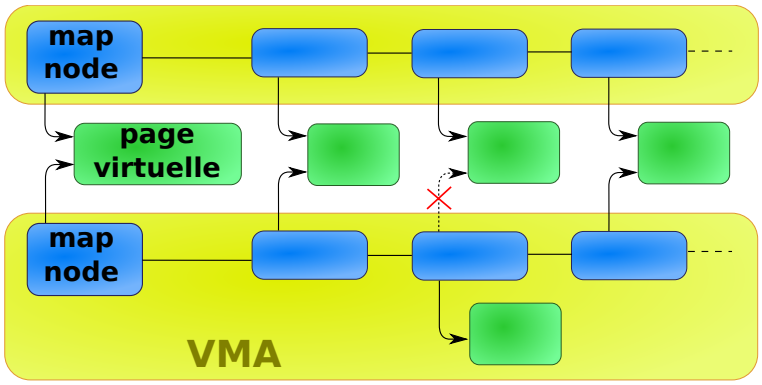

FIGURE 3.5.: *Exemple de copie à l'écriture de pages partagées par deux processus.*

1. Recherche de la page correspondant à la faute dans l'arbre.
2. Si aucune page n'est trouvée, il s'agit d'un artéfact lié au multi-thread. Relancer l'erreur corrige le problème. Ce dernier apparaît lorsque plusieurs fautes d'écritures surviennent en même temps qu'une faute de page. Dans ce cas l'ordre de verrouillage des mutex laisse la place à la mise en attente de quelques threads dans une mauvaise section du code, produisant ce "bug". L'erreur n'est pas corrigible en laissant tous nos mutex se mettre en place dans les méthodes évènementielles des VMA. Une détection de boucle infinie a été mise en place pour détecter un éventuel problème plus sévère dans cette portion du module.

3. Vérification du compteur atomique de la page virtuelle. S'il est supérieur à un, alors la page est partagée et l'on doit appliquer une copie avant d'autoriser l'écriture.
4. Si le compteur est à un, la page virtuelle n'est pas partagée et l'écriture peut être autorisée.

Au niveau de notre description des associations, l'utilisation de la structure *vhp_map_node* pour accrocher les pages virtuelles aux VMA nous permet de traiter la copie sur écriture simplement comme cela est montré sur la figure 3.5.

On remarquera que la faute d'écriture sera produite pour chaque page standard et non pas une seule fois par page virtuelle comme c'est le cas pour la faute de page. Cette méthode est donc certainement l'un des points critiques du module en ce qui concerne les performances.

3.10. Support d'un mode coloration de pages

Il est clair que l'implémentation du support des pages virtuelles est complexe à mettre en place et sujet à beaucoup de questions. Afin de disposer d'une base de comparaison j'ai proposé pendant mon stage une rapide implémentation d'un mode utilisant la coloration de page, toujours en tant que module.

Le support de ce mode a été réalisé au milieu de l'implémentation des pages virtuelles afin d'en réutiliser les structures et ne pas perdre de temps à fournir une nouvelle implémentation depuis le début. Les structures de VHP ne sont pas optimales pour cet usage mais suffisent pour effectuer quelques tests et pour comparer les difficultés d'implémentation.

L'utilisation de la coloration de pages nécessite la présence dans le noyau d'une liste de pages libres pour chaque couleur utilisée. Ne pouvant modifier le cœur du noyau, nous fournirons ces listes via un cache de pages triées par ligne de couleur et maintenu par le module. Cette approche est consommatrice de mémoire mais permet de mettre en place l'approche depuis un module en toute simplicité. Lorsqu'une liste du cache sera vidée, nous remplirons le cache en demandant un nombre déterminé de pages au système. Ces pages seront alors triées et placées dans les différentes listes du cache.

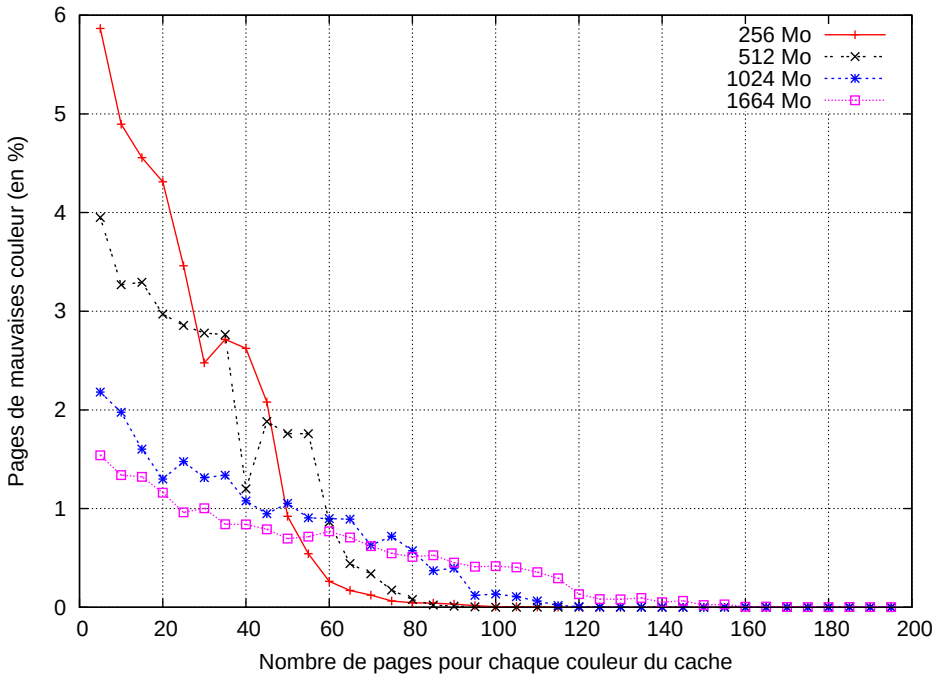

FIGURE 3.6.: *Taux d'erreurs d'associations de couleur en fonction de la taille du cache de page et de la taille de la VMA à allouer. Le test a été réalisé sur une machine disposant de 2 Go de mémoire.*

Pour un cache L2 de 2 Mo avec des pages de 4 Ko et 8 voies, on peut calculer qu'il nous faut utiliser :

$$\frac{2 * 1024}{8 * 4} = 64 \tag{3.1}$$

couleurs pour éliminer les collisions. En allouant 120 pages par couleur pour ce cache, on utilise un total de 30 Mo ce qui reste raisonnable. La limite du nombre de pages du cache nous est donnée par la figure 3.6. On pourra améliorer le système en permettant au cache de se réduire si le système n'a plus assez de mémoire. On notera que le support du mode "copie sur écriture" n'a pas encore été implémenté pour la coloration de pages.

3.11. Support du swap

Lorsque le système n'a plus assez de mémoire il peut utiliser une partition du disque dur pour déplacer les pages peu utilisées. Après quelques recherches sur ce point il s'avère qu'il n'est pas possible de supporter le swap pour des pages composées, telles qu'elles sont utilisées par notre module. Ce point n'est pas critique pour des applications de calcul scientifiques qui se doivent d'éviter d'arriver à cette situation.

On note toutefois une difficulté concernant ce point. Le non-support du swap pour la mémoire allouée par notre module peut conduire le système à la faute. Par divers essais on parvient à observer trois comportements. Ces trois comportements sont décrits dans ce qui suit en considérant une application utilisant notre module et tentant d'allouer plus de mémoire que la machine n'en dispose.

Swap désactivé : Si l'hôte ne dispose pas de partition ou fichier de swap, Linux arrivant à saturation va tenter de tuer le processus le plus gourmand en mémoire. Dans ce cas, l'application est correctement identifiée et tuée par Linux, comme si elle utilisait *malloc* et sa mémoire est libérée.

Swap activé : Ce cas est plus critique, Linux tente de libérer de la mémoire en cherchant des pages candidates au swap. Nos pages n'entrant pas dans cette catégorie, ce sont essentiellement les pages des autres processus, puis de l'OS lui-même qui sont choisies. Arrivé à un certain stade, Linux se voit obligé de swaper puis récupérer en boucle certaines pages pour satisfaire ses traitements, et entre dans un cycle bloquant lié à la lenteur des accès disques. On évite ce cas dans le module en arrêtant de fournir de la mémoire aux processus dès que la mémoire disponible en RAM atteint une valeur critique. On notera qu'il reste à trouver une manière de déterminer ce seuil en fonction de la machine hôte. On pourrait envisager de fournir des pages standards candidates au swap dès que la limite est dépassée, plutôt que renvoyer une erreur d'épuisement de la mémoire.

Pas d'allocation au premier accès : Ce cas avait été testé en début d'implémentation. Bien qu'il ne soit plus utilisé, il est important de noter que ce fonctionnement conduit à un *kernel panic*[5]. Le problème se pose si notre application est en train d'allouer de la mémoire quand cette dernière arrive à l'épuisement. Comme le module tente d'allouer toute la mémoire d'un coup, il ne laisse pas le temps au démon de swap d'intervenir, conduisant rapidement l'OS à ne plus disposer de la mémoire nécessaire au moindre traitement. L'implémentation finale de notre module ne doit plus produire ce type d'effet en utilisant d'autres options d'allocations, mais ce cas reste à vérifier si l'on désire modifier le module pour allouer toute la mémoire au moment du *mmap*.

3.12. Intégration dans les programmes cibles

Maintenant que notre module permet d'allouer de la mémoire en utilisant des pages virtuellement plus grosses, il faut parvenir à intégrer ce comportement dans les applications du CEA. On peut aborder cette intégration de deux manières. Soit l'application est modifiée pour utiliser *mmap* sur

5. Lorsque le noyau détecte que quelque chose d'anormal est en train d'arriver et qu'il n'a aucune instruction pour corriger la situation, il entre dans le mode kernel panic. Dans ce cas Linux affiche quelques informations de débogages et arrête brutalement son exécution.

notre périphérique. Dans ce cas elle sera spécifiquement optimisée pour utiliser notre module. Soit on fournit un nouvel allocateur que l'on pourra utiliser dans les applications grâce à l'utilisation de la variable d'environnement *LD_PRELOAD*.

LD_PRELOAD permet de forcer le chargement d'une bibliothèque dynamique au lancement d'une application. Ceci permet de remplacer certaines fonctions de l'application ou des bibliothèques qu'elle utilise sans recompilation. On pourra aussi utiliser l'éditeur de lien pour lier notre allocateur statiquement à la compilation. Dans les deux cas, on ne modifie pas le code source.

Afin d'éviter de rentrer dans les problèmes liés à l'implémentation d'un allocateur efficace, nous avons modifié l'allocateur de la bibliothèque MPC [6] [15] développée au CEA afin qu'il puisse utiliser notre module (annexe A). De manière à disposer d'un élément de test plus simple à maîtriser nous avons également implémenté un allocateur simpliste que nous pouvons manipuler plus facilement pour obtenir les comportements souhaités. Dans la suite nous nommerons cet allocateur *vhpalloc*. On fournira trois versions dérivées : *vhpalloc-anonyme* pour les pages standards anonymes, *vhpalloc-color* pour la coloration de page et *vhpalloc-huge* pour les *huge pages* de Linux.

3.13. Multiplication des VMA et appel à ioctl

Une analyse détaillée du fonctionnement de l'allocateur de MPC montre que ce dernier effectue des appels successifs à *mmap* pour obtenir la mémoire dont il a besoin. De même, l'allocateur utilise *munmap* pour libérer les blocs de mémoire de taille supérieure à 2 Mo. On peut donc se poser la question du problème d'augmentation du nombre de VMA dans le processus.

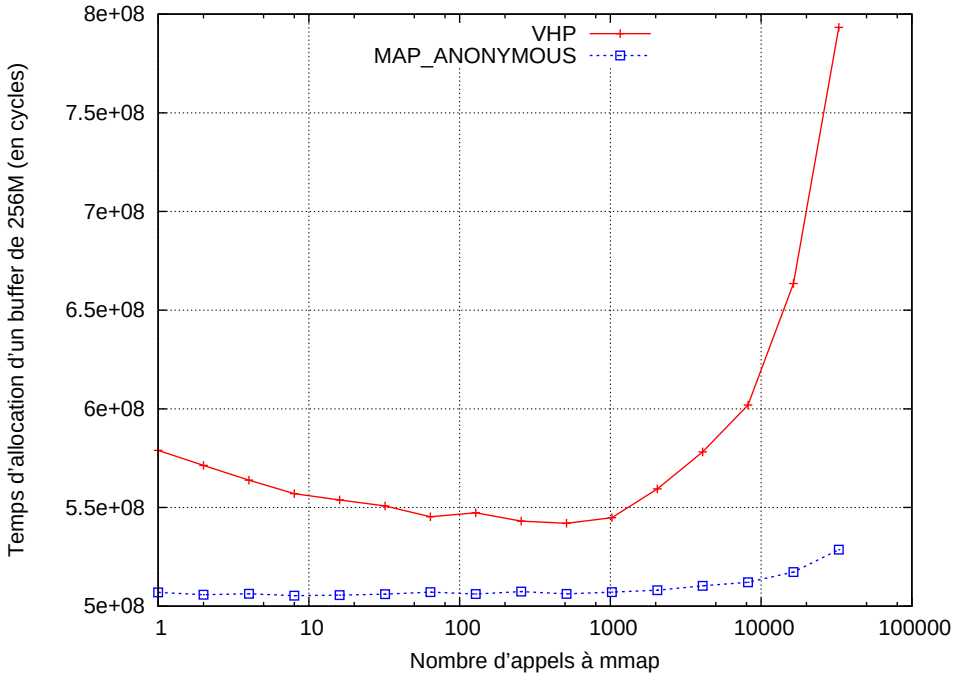

FIGURE 3.7.: *Augmentation du temps d'allocation d'un buffer de 256 Mo en utilisant un nombre croissant de VMA.*

On remarque en effet que l'implémentation de la gestion des VMA du noyau passe par des listes et donc un fonctionnement en $O(N)$. L'utilisation d'un millier de VMA n'est pas un problème mais une forte augmentation de ce nombre pourrait devenir pénalisante pour l'allocateur. Lorsqu'une VMA est créée à côté de VMA similaires, le noyau va prendre en charge une fusion de ces dernières.

6. MPC (MultiProcessor Communication) est dédiée à la programmation parallèle pour le HPC. Elle offre une implémentation multi-thread orientée NUMA de la norme MPI et son propre support des threads POSIX. Cette bibliothèque est en grande partie développée par Marc PÉRACHE (mon encadrant de stage) et Patrick CARRIBAULT.

En se limitant au mode *MAP_ANONYMOUS*, MPC maintient donc un nombre certes élevé de VMA, mais limité par les fusions.

Or, on observe que le noyau ne fournit plus ce comportement dès lors que les VMA définissent leur propre méthode *close*, ce qui est notre cas. Notre approche risque donc de conduire l'allocateur à créer un nombre croissant de VMA qui finira par pénaliser l'application si elle tourne longtemps en effectuant beaucoup d'allocations et libérations. La figure 3.7 montre l'augmentation du temps d'allocation avec le nombre de VMA avec ou sans fusion.

On remarque sur la figure une forte augmentation du temps d'allocation avec le nombre de VMA pour le mode VHP, comportement qui est largement réduit en mode *MAP_ANONYMOUS*. On observe également un comportement décroissant pour un faible nombre de VMA. Ce comportement est lié à une liste encore non remplacée par le RB-tree dans la méthode *vhp_vm_fault*. Cette liste devra disparaître dans la suite, mais a été gardée dans un premier temps afin tester la stabilité du module et détecter certains bugs liés à l'introduction du RB-Tree dans le module.

Bien qu'il reste à prouver que les applications conduisent à ce type de cas, nous avons proposé une solution en utilisant l'appel système *ioctl*[7] pour demander la libération d'un segment de mémoire de nos VMA sans supprimer ou couper ces dernières. Pour ce faire nous supprimons simplement les associations de pages du segment à l'aide de l'appel à *zap_vma_ptes*. Cette approche n'a pas encore été intégrée à MPC mais fonctionne avec *vhpalloc* en utilisant *mremap* pour augmenter la taille des VMA.

3.14. Configuration et suivi du module

L'administrateur de l'hôte peut configurer le module pour adapter les paramètres du tableau 3.1 depuis le dossier système */sys/module/vhp/parameters/* ou au chargement du module.

max_page_order	Définie la taille des pages virtuelles.
color_mode	0 pour le mode *hachage* et 1 pour le mode *tourniquet*
color_cache_size	Nombre de pages par couleur pour le cache.

TABLE 3.1.: *Options du module accessibles via /sys/module/vhp/parameters*

On pourra également utiliser le fichier */proc/vhp_stats* pour récupérer quelques statistiques listées dans le tableau 3.2. Afin de faciliter certains débogages, nous pouvons aussi obtenir la liste des pages virtuelles utilisées par le module via le fichier */proc/vhp_map*.

unmap_leak	Non implémenté pour l'instant, il permet normalement d'évaluer la quantité de mémoire gâchée par les découpes des VMA en mode pages virtuelles.
nb_smaller	Renvoie le nombre de pages allouées n'ayant pas la taille demandée par *max_page_order*.
another_color	Compte le nombre de pages fournies avec une mauvaise couleur en mode coloration de page.
total_color_pages	Compte le nombre total de pages allouées en mode coloration de page.
cache_size	Donne la taille actuelle du cache de couleur.

TABLE 3.2.: *Variables exportées par le fichier de statistique de fonctionnement du module /proc/vhp_stats.*

7. L'appel *ioctl* est un appel système générique permettant de dialoguer de manière moins contrainte avec les périphériques. Les arguments sont libres et non vérifiés par le noyau.

4. Résultats

Maintenant que nous avons étudié en détail la solution proposée, nous allons la tester sur différents types d'applications pour évaluer notre impact sur ces dernières. On précise que tous les tests qui suivent ont été réalisés sous des systèmes d'exploitation 64 bits.

4.1. Boucles sur un tableau

Nous avons vu en section 2.4 et 2.5 que l'on pouvait mettre en évidence une fuite des caches sous Linux et que l'utilisation des *huge pages* permettait de corriger le problème. La figure 4.1 montre l'impact de notre approche sur le code 2.6.

On remarque que l'utilisation de VHP avec des pages virtuelles de 256 Ko permet d'obtenir les mêmes résultats que les *huge pages*. De même que l'utilisation de la coloration de pages avec 64 couleurs. En pratique l'utilisation de pages virtuelles de 256 Ko correspond exactement à l'utilisation de 64 couleurs ; on a en effet $64 * 4$ Ko $= 256$ Ko pour les pages standards de 4 Ko de *x86*. On peut donc bien utiliser des pages virtuelles plus petites que 2 Mo pour éviter la fuite de cache. On remarquera qu'il serait presque possible d'utiliser des pages de 128 Ko avec des résultats proches des *huge pages*. Dans ce qui suit nous utiliserons des pages de 256 Ko afin d'être certains de se situer dans le cas idéal.

Les deux approches permettent donc bien de résoudre le problème de fuite des caches mis en évidence par la simple boucle du code de la figure 2.6. Nous allons voir dans ce qui suit si l'impact est réel sur des applications plus complexes.

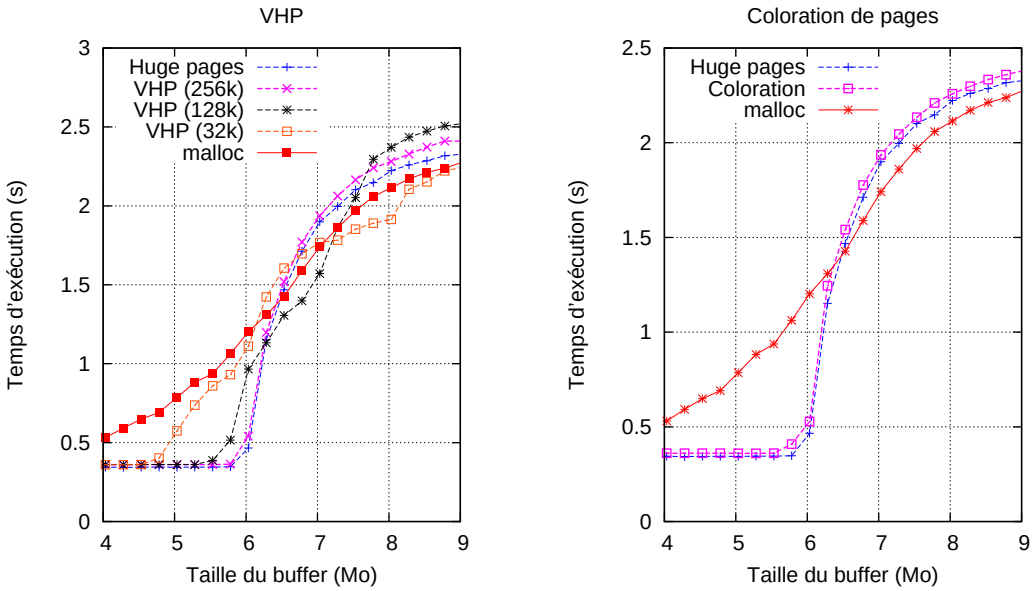

FIGURE 4.1.: *À gauche, temps d'exécution du code de la figure 2.6 en utilisant VHP pour différentes tailles de pages virtuelles. À droite en utilisant de la coloration de pages avec 64 couleurs. Le test est réalisé sur un Xeon Core2 Quad d'Intel avec de 6 Mo de cache L2.*

4.2. Temps d'allocation

Notre module fournit l'allocation via un mécanisme supplémentaire ; on peut donc se demander si cela ne pénalise pas trop le temps d'allocation. Le tableau 4.1 donne les temps d'allocation de tableaux de différentes tailles avec la libc ou VHP. On remarque une augmentation moyenne de 4,7% du temps d'allocation avec un premier accès en écriture, et une diminution de 2,8% pour un premier accès en lecture, l'écart étant à peu près constant en fonction de la taille du buffer à allouer.

Notre système d'allocation est plus rapide sur les fautes de pages mais plus lent sur les fautes d'écriture. Cette différence s'explique par le fonctionnement différent au niveau du noyau. La faute de page n'est produite qu'une fois par page virtuelle alors que la faute d'écriture est produite une fois par page standard. Dans le premier cas, la réduction du nombre d'appels induits par l'utilisation de pages plus grosses compense le surcoût apporté par les opérations de VHP. Le surcoût de l'accès en écriture peut être annulé en désactivant la copie sur écriture, c'est donc le support de cette dernière qui nous coûte le plus cher en temps d'allocation.

	Lecture			Écriture		
Taille (Mo)	libc	VHP	Différence (%)	libc	VHP	Différence (%)
16	172,06	166,11	-3,5	150,49	157,88	4,9
64	686,67	666,43	-2,9	600,5	622,6	3,7
128	1366,9	1325,4	-3,0	1191,1	1236,3	3,8
256	2718,8	2645,6	-2,7	2371,9	2491,7	5,1
512	5422,9	5310,1	-2,1	4732,5	4982	5,3
1024	10874	10614	-2,4	9456,2	10018	5,9

TABLE 4.1.: *Temps d'allocation moyen pour différentes tailles de buffer en utilisant la libc ou VHP. La mémoire est allouée puis accédée en lecture ou en écriture. Les temps sont donnés en millions de cycles.*

4.3. Linpack

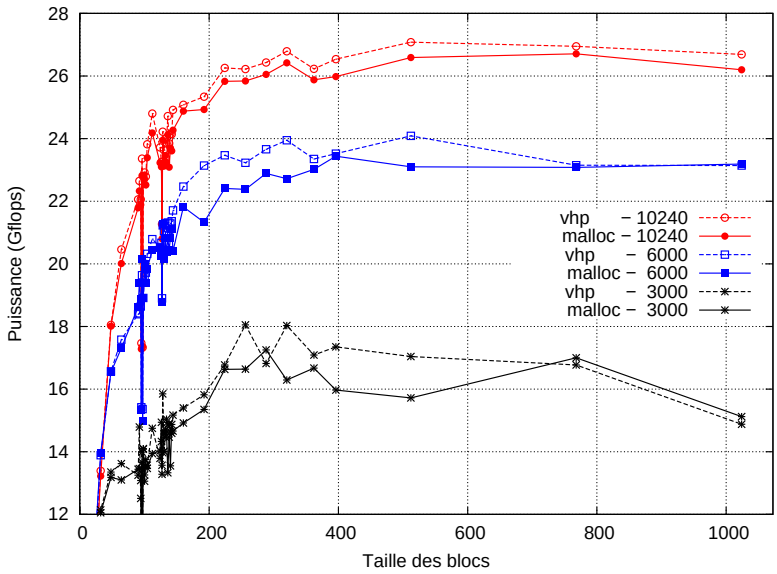

FIGURE 4.2.: *Résultats de mesure de performance d'un Xeon Core 2 Quad d'Intel avec le Linpack et la bibliothèque ATLAS sur un seul processus MPI pour les problèmes de taille 3000, 6000 et 10240.*

Le *Linpack* est l'application utilisée par le projet Top 500 pour établir son classement des plus puissants supercalculateurs mondiaux. Il est donc intéressant de voir si notre solution permet de gagner des performances sur ce type d'application.

Le Linpack résout un problème linéaire de n équations à n inconnues en utilisant la méthode du pivot de Gauss. Les multiplications de matrices nécessaires à cette résolution sont implémentées à l'aide la bibliothèque BLAS [1] [16] qui est à l'origine d'une bibliothèque plus complète : LAPACK[17]. On trouve plusieurs implémentations de BLAS ; on citera entre autres ATLAS [2][18] qui est une version optimisée de BLAS. La MKL [3] est une bibliothèque similaire implémentée par Intel et totalement optimisée pour leurs processeurs. Pour nos tests sur le *Linpack* nous testerons ces deux bibliothèques.

L'utilisation du *Linpack* passe par des choix importants de configuration du problème à résoudre pour trouver les paramètres utilisant au mieux la machine. Comme nous cherchons à observer l'impact de notre module sur l'application, nous prendrons une plage de valeurs afin d'observer un éventuel changement de comportement dans des secteurs particuliers. Pour la taille du problème nous utiliserons les valeurs : 1000, 2000, 3000, 6000, 10240, 17920 et 19328 ; 19328 correspondant au maximum atteignable sur une machine disposant de 4 Go de mémoire.

Parmi les paramètres du *Linpack*, nous utiliserons principalement le choix de taille de problème et la taille des blocs de traitement. Disposant d'un Xeon Core 2 Quad d'Intel, nous ferons les tests avec un ou quatre processus MPI. N'étant pas limités par des performances réseau nous utiliserons *OpenMPI*[4] pour lancer les tâches parallèles sur la machine.

Les courbes complètes des performances obtenues pour les tailles 3000, 6000 et 10240 sont données dans la figure 4.2. Les gains obtenus sur les maximas pour les différents problèmes sont représentés sur la figure 4.3. Un fichier complet de notre configuration *Linpack* est fourni en annexe B.

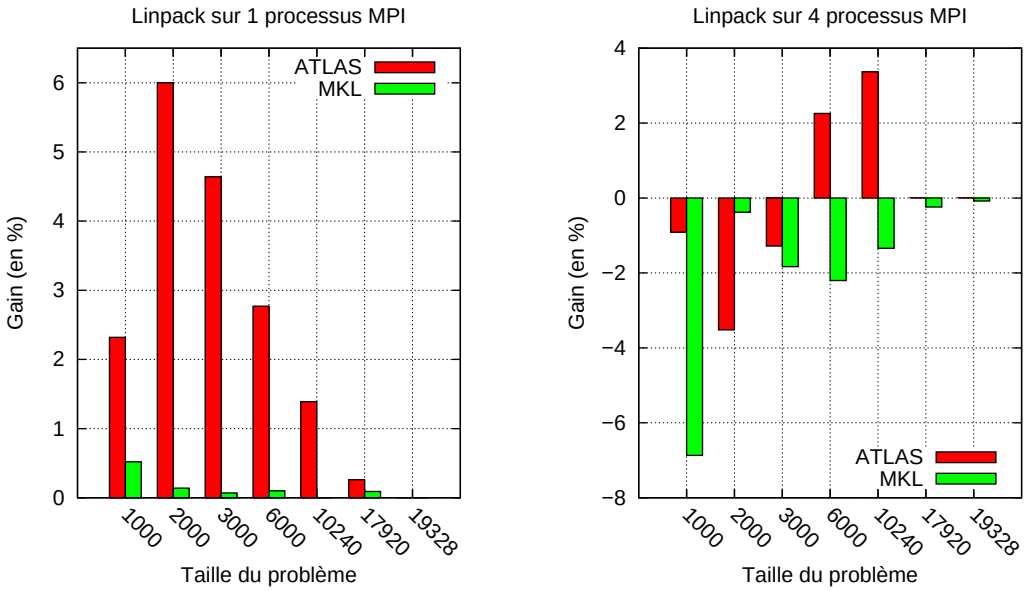

FIGURE 4.3.: *Gains observés en cherchant la performance maximum renvoyée par Linpack pour les différentes configurations en utilisant 1 ou 4 processus MPI. Les résultats détaillés sont donnés dans l'annexe C*

On observe bien un gain dans certaines conditions sur le *Linpack* : jusqu'à 6% en utilisant ATLAS avec un seul processus pour des problèmes de petites tailles. Il est bien plus difficile de

1. Basic Linear Algebra Subprograms
2. Automatically Tuned Linear Algebra
3. Math Kernel Library
4. Message Passing Interface : il s'agit de bibliothèques utilisées en HPC pour distribuer des tâches sur plusieurs nœuds et prendre en charge les communications entre les différents processus.

gagner des performances avec la MKL qui est déjà très proche de la puissance crête du processeur et certainement optimisée pour utiliser au maximum le cache L1. Notre impact dépend de la taille du problème, nous sommes donc performants dans certains cas de figures seulement.

4.4. Applications à usage réel

Bien que le *Linpack* soit une application importante pour le monde du HPC, celle-ci n'est pas totalement représentative. Nous allons maintenant effectuer nos tests sur plusieurs applications réelles développées au CEA/DAM Île-de-France.

4.4.1. Magnétohydrodynamique 2D

L'un des doctorants du laboratoire d'accueil (Marc Wolff) travaille sur un code de calcul de type MHD [5], opérant sur maillages réguliers cartésiens avec des schémas numériques d'ordre très élevés (schémas de type Godunov d'ordre 2, 3 ou 4). Nous allons utiliser ce code de calcul comme support de test. Le fichier de configuration pour réaliser ce test est donné dans l'annexe D. Le tableau 4.2 donne les temps d'exécutions de cette application.

OS	Allocateur	gcc	Temps (s)	Écart à la libc Linux (%)
Linux	libc	3.4	887,78	15,3
	libc	4.3	770,06	0
	vhpalloc	4.3	774,98	0,6
	vhpalloc-ano	4.3	770,3	0,03
	MPC	4.3	1017,04	32,1
	MPC-VHP	4.3	1041,70	35,3
OpenSolaris	MPC	3.4	972,68	26,3
	libc	3.4	720,81	-6,4
FreeBSD 6.4	libc	3.4	953,85	23,9
FreeBSD 7.2	libc	4.3	1069,25	38,8

Table 4.2.: *Temps d'exécution de l'application MHD avec une grille 400*400 dans différents contextes d'allocation.*

En observant les résultats de la version originale dans les différents contextes nous sommes surpris par la grande variabilité (jusqu'à 35% d'écart à la libc Linux) des temps obtenus. En observant le code de plus près il s'avère que certaines méthodes d'interpolation de l'application provoquent beaucoup plus de fautes de pages avec l'allocateur MPC qu'avec celui de la libc. Or après quelques corrections il ne restait plus d'allocations au milieu de l'exécution. La différence provient donc de la manière dont la mémoire est allouée et non du temps d'allocation.

En effectuant quelques analyses des adresses renvoyées par les différents allocateurs on peut remarquer que leur distribution n'est pas tout à fait la même. La figure 4.4 montre la distribution des adresses renvoyées par les différentes implémentations de malloc, en y appliquant un modulo 64 (correspondant à la taille d'une ligne de cache). On remarque clairement une différence. La libc maintient des adresses alignées sur 16 octets, l'allocateur MPC quant à lui renvoie uniquement des adresses alignées sur 64 octets et *vhpalloc* fait une sorte de compromis entre les deux avec une distribution biaisée.

En jouant sur la distribution des adresses à l'aide d'offsets aléatoires sur les pointeurs renvoyés on parvient à faire varier du simple au double le temps d'exécution de la méthode concernée. Le problème vient donc bien de la manière d'aligner les données en mémoire. Il est toutefois étonnant que le pire cas soit celui de MPC alors que son allocateur reste aligné sur les lignes de cache.

Afin d'effectuer quelques tests nous avons tenté de découper le travail de la méthode autrement. Au lieu de travailler sur tous les tableaux en même temps nous avons essayé de découper les boucles, en espérant obtenir une meilleure optimisation de la part du compilateur. Les temps d'exécutions

5. MagnétoHydroDynamique

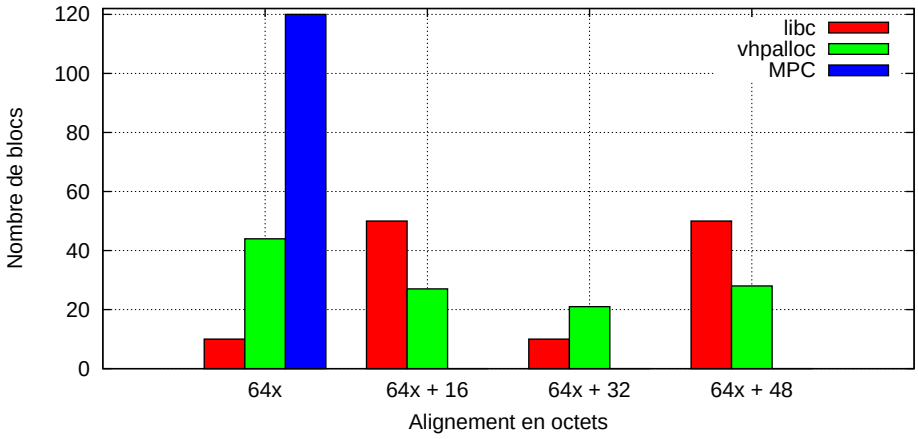

FIGURE 4.4.: *Distribution des alignements mémoires renvoyés à l'application MHD par les différentes implémentations de malloc.*

de cette version corrigée sont donnés dans le tableau 4.3. On obtient bien une meilleure stabilité bien que les temps d'exécutions soient en moyenne plus longs.

OS	Allocateur	gcc	Temps d'exécution (s)	Écart à la libc Linux (%)
Linux	libc	3.4	898,4	15,3
	libc	4.3	779,37	0
	vhpalloc	4.3	891,73	14,4
	vhpalloc-ano	4.3	889,37	14,1
	MPC	4.3	924,99	18,7
	MPC-VHP	4.3	928,94	19,2
OpenSolaris	MPC	3.4	630,64	-19,1
	libc	3.4	601,68	-22,8
FreeBSD 6.4	libc	3.4	918,05	17,8
FreeBSD 7.2	libc	4.3	836,92	7,4

TABLE 4.3.: *Temps d'exécution de l'application MHD modifiée avec une grille 400*400 dans différents contextes d'allocation.*

On obtient également des résultats très variables d'un OS à l'autre, OpenSolaris donnant systématiquement les meilleurs résultats. Il reste donc à comprendre en détail pourquoi une telle différence est observée en changeant la méthode d'allocation. Pour pouvoir utiliser correctement VHP nous devrons parvenir à comprendre et mieux maîtriser ces variations.

4.4.2. Transfert radiatif PN

L'un des stagiaires du laboratoire travaille sur la parallélisation d'un code de calcul de transfert radiatif. Il s'agit d'un code 2D instationnaire opérant sur maillage cartésien, l'intensité radiative étant décomposée sur la base des harmoniques sphériques (schéma de type Godunov *asymptotic preserving diffusion* d'ordre 1). Nous allons pouvoir prendre cette application comme base de test pour notre module. Le tableau 4.4 donne quelques résultats obtenus avec cette application.

On remarque à nouveau le même genre de problème qu'avec le code MHD en utilisant l'allocateur MPC mais moins marqué. Avec un seul processus notre module réduit encore les performances. On note toutefois un gain de 1.4% sur le temps total par rapport à l'utilisation de l'allocateur MPC dans le test à 4 processus. Ici MPC offre un gain de performances sur le temps des produits de matrices.

Processus	allocateur	Temps total (s)	Temps des produits de matrices (s)
1	libc	394	294
	MPC	407	279
	MPC-VHP	408	292
4	libc	468	300
	MPC	486	285
	MPC-VHP	479	285

TABLE 4.4.: *Résultats d'un test de résolution de problème avec N=15 à 1 ou 4 processus en utilisant la MKL avec le code PN. Avec un seul processus on utilise t=0.0001, avec quatre on utilise t=0.001, où t est le temps final de la simulation en secondes.*

4.4.3. bzip2

Nous avons appliqué notre solution sur *bzip2* en compressant les sources du noyau avec l'option "-9". Le graphique 4.5 donne les temps moyens sur 8 exécutions de compression pour les différents modes d'allocation. Sur cette application on observe un gain de 0,9% avec l'approche VHP. On remarque que l'utilisation des *huge pages* donne un résultat bien meilleur que nos pages virtuelles, probablement lié au problème des TLB qui n'est pas résolu par notre solution. Cette supposition reste à vérifier à l'aide des compteurs du processeur. On note également que la coloration de pages donne des résultats proches de VHP, notamment le mode *hachage*.

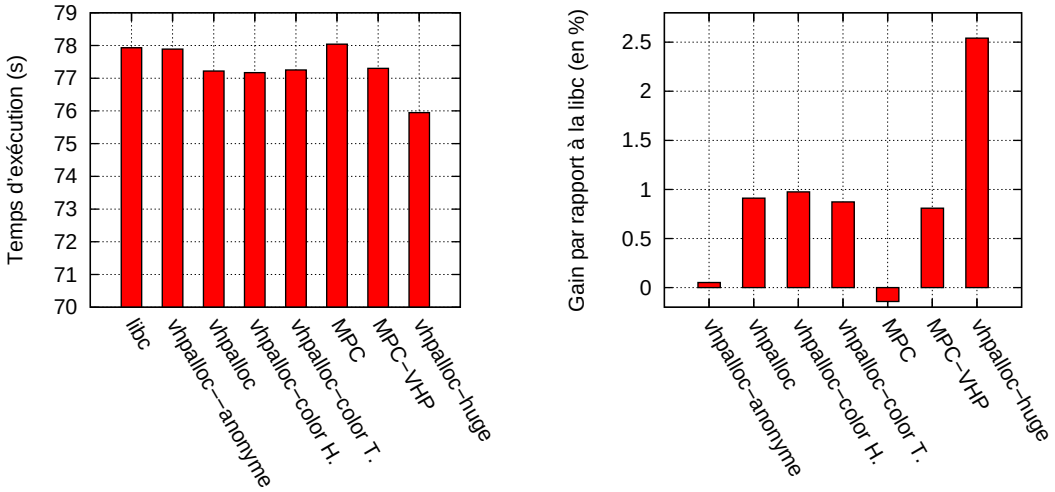

FIGURE 4.5.: *Résultats d'application de la solution sur bzip2 en compressant les sources du noyau. Le test est effectué avec les allocateurs : libc, vhpalloc-anonyme (en utilisant des pages standards), vhpalloc, vhpalloc-color (en utilisant les modes "hachage" ou "tourniquet") et vhpalloc-huge pour les huge pages. Le test est réalisé sur un Core 2 Duo D'Intel disposant de 2 Mo de cache.*

4.5. Sensibilité aux allocateurs

Nous avons vu avec les applications MHD et PN que les cas réels étaient beaucoup plus complexes. Entre autres, il a été mis en évidence une sensibilité très importante de ces applications à la manière dont l'allocateur renvoie ses adresses. Des gains ont toutefois été observés dans certaines conditions, il est donc nécessaire de creuser plus en avant cette question afin d'arriver à mieux utiliser notre solution. Les gains observés sous OpenSolaris sont également intéressants et nécessitent un approfondissement.

Conclusion et ouverture

Nous avons vu dans la première partie que l'utilisation de caches de type associatifs à N voies nécessitait une attention particulière de la part du système d'exploitation pour limiter les conflits de lignes de cache. Nous avons également mis en évidence une faiblesse de Linux sur ce point par rapport à OpenSolaris et FreeBSD. Nous avons vu qu'il était possible de corriger le problème sous Linux en utilisant des pages plus grosses via les *huge pages*, mais que ces dernières n'étaient pas exploitables dans le contexte HPC.

Nous avons au cours de ce stage proposé un module noyau permettant de compenser ce problème de fuite des caches. La solution est orientée vers l'utilisation de pages virtuellement plus grosses (VHP) composées de pages standards de 4 Ko. N'étant pas dépendants du matériel, nous avons pu utiliser des pages de 256 Ko pour être moins gênés par la fragmentation de la mémoire.

Nous avons réussi à contourner la plupart des problèmes s'opposant à ce type d'implémentation et aboutir à un module fonctionnel intégrable dans les applications via une bibliothèque dynamique fournissant *malloc*. Le module permet également d'utiliser un mode de coloration de pages pour servir de comparaison, même si ce mode ne supporte pas encore tous les appels pris en charge par VHP.

Après avoir implémenté les deux méthodes, on peut conclure que le support de la coloration de pages représente une implémentation beaucoup plus simple à mettre en place dans un module. Si les performances s'avèrent similaires aux pages virtuelles il serait judicieux de se tourner plutôt vers la coloration de pages qui sera plus simple à maintenir dans le temps.

Les résultats de la section 4.1 montre que VHP permet de résoudre le problème de fuite du cache sur la simple boucle utilisé pour mettre le problème en évidence. En utilisant notre solution sur différentes applications nous avons pu observer quelques gains, jusqu'à 6% sur certaines configurations du *Linpack* en utilisant ATLAS, ou 0,9% sur bzip2. L'impact des allocateurs utilisés reste à étudier afin de parvenir à utiliser efficacement le module.

Concernant VHP, certains supports restent à implémenter, on notera entre autre le support du mode *MAP_SHARED*, un choix correct de limite de mémoire libre pour régler le problème de swap ainsi qu'un support explicite des architectures de types NUMA. Concernant l'intégration, il reste un fichier de configuration à fournir pour *udev*. On notera que le code développé est compatible avec les versions 2.6.27 à 2.6.30 du noyau. On peut proposer la rédaction d'un correctif mineur permettant d'exporter les méthodes d'utilisation des *huge pages* pour les intégrer dans notre approche si les VMA de Linux peuvent supporter les deux types de pages simultanément.

Les résultats obtenus sont à confirmer sur les processeurs de type *Nehalem* d'Intel, avec l'ajout d'un cache L3 et une bande passante mémoire plus importante.

Annexes

A. Correctif de l'allocateur MPC

```
--- MPC_Allocator/src/sctk_alloc.c 2009-06-23 16:49:30.000000000 +0200
+++ MPC_Allocator/src/sctk_alloc.c 2009-07-31 17:05:08.000000000 +0200
@@ -23,6 +23,7 @@
 #define SCTK_SKIP_SIZE ((unsigned long)1024 * 1024 * 1024)
 #define SCTK_BOTTOM_SKIP ((unsigned long)0x3ce00000)
 #define SCTK_TOP_SKIP (SCTK_BOTTOM_SKIP + SCTK_SKIP_SIZE)
+#define SCTK_USE_VHP

 #include "sctk_config.h"
 #include "sctk_alloc_api.h"
@@ -122,12 +123,34 @@
 sctk_mmap (void *ptr, sctk_size_t size)
 {
   void *tmp;
+  #ifdef SCTK_USE_VHP
+  static int fd = -1;
+  if (fd == -1)
+  {
+ const char * vhp_fname = getenv("VHP_FILENAME");
+ if (vhp_fname==NULL || *vhp_fname=='\0')
+ vhp_fname = "/dev/vhp";
+ fd = open(vhp_fname, O_RDWR, 0755);
+ if (fd < 0)
+ perror("Open failed, check your VHP module intallation");
+  }
+  assume (fd >= 0);
+  #endif
   SCTK_TRACE_START (sctk_mmap, ptr, size, NULL, NULL);
   SCTK_DEBUG (sctk_mem_error
        ("PREPARE MMAP %p-%p %lu\n", ptr, (char *) ptr + size, size));
+  #ifdef SCTK_USE_VHP
+  tmp = mmap ((void *) (ptr), (size),
+          PROT_READ | PROT_WRITE,
+          MAP_PRIVATE | MAP_FIXED, fd, 0);
+  #else
   tmp = mmap ((void *) (ptr), (size),
        PROT_READ | PROT_WRITE,
        MAP_PRIVATE | MAP_FIXED | MAP_ANONYMOUS, -1, 0);
+  #endif
   if (ptr != tmp)
     {
       perror ("MMAP FAIL");
```

B. Configuration du Linpack

```
HPLinpack benchmark input file
Innovative Computing Laboratory, University of Tennessee
HPL-1.out      output file name (if any)
6            device out (6=stdout,7=stderr,file)
4            # of problems sizes (N)
1000 2000 3000 6000 Ns
18            # of NBs
32 48 64 90 92 94 96 98 100 102 104 108 112 116 120 124 126 128   NBs
0            PMAP process mapping (0=Row-,1=Column-major)
1            # of process grids (P x Q)
1            Ps
1            Qs
16.0         threshold
1            # of panel fact
2            PFACTs (0=left, 1=Crout, 2=Right)
1            # of recursive stopping criterium
4            NBMINs (>= 1)
1            # of panels in recursion
2            NDIVs
1            # of recursive panel fact.
1            RFACTs (0=left, 1=Crout, 2=Right)
1            # of broadcast
1            BCASTs (0=1rg,1=1rM,2=2rg,3=2rM,4=Lng,5=LnM)
1            # of lookahead depth
1            DEPTHs (>=0)
2            SWAP (0=bin-exch,1=long,2=mix)
64           swapping threshold
0            L1 in (0=transposed,1=no-transposed) form
0            U  in (0=transposed,1=no-transposed) form
1            Equilibration (0=no,1=yes)
8            memory alignment in double (> 0)
##### This line (no. 32) is ignored (it serves as a separator). ######
0                               Number of additional problem sizes for PTRANS
1200 10000 30000                values of N
0                               number of additional blocking sizes for PTRANS
40 9 8 13 13 20 16 32 64        values of NB
```

C. Résultats du Linpack

Les tableaux C.1 et C.2 donnent les résultats numériques obtenus avec le *Linpack*. On remarquera qu'ATLAS donne étrangement de meilleurs résultats que la MKL avec un seul processus. Ceci est lié au fait qu'ATLAS utilise automatique plusieurs threads lorsque c'est utile. Avec quatre processus, la MKL repasse bien devant ATLAS. ATLAS utilise plus de mémoire que la MKL, et nous avons utilisé notre allocateur simpliste pour les tests. Ce dernier faisant plus de gâchis mémoire nous n'avons pas pu faire fonctionner les plus gros problèmes avec ATLAS et VHP. On notera que le couplage ATLAS/MPC/Linpack donne lieu à une explosion de l'utilisation de la mémoire suivant la façon dont ils sont liés.

Le benchmark a été réalisé sur un Xeon Core 2 Quad Q9550 dont la puissance théorique donnée par Intel est 45.28 GFlops [19].

	Utilisation d'un seul processus MPI.					
	ATLAS			MKL		
Problème	malloc	vhp	Gain (%)	malloc	vhp	Gain (%)
1000	9,056	9,266	2,32	9,099	9,146	0,52
2000	14,17	15,02	6	9,341	9,354	0,14
3000	17,25	18,05	4,64	9,762	9,769	0,07
6000	23,44	24,09	2,77	10,27	10,28	0,1
10240	26,71	27,08	1,39	10,37	10,37	0
17920	30,49	30,57	0,26	10,58	10,59	0,09
19328	31,15			10,72	10,72	0

Table C.1.: *Résultats des maximums obtenus pour les différents tests réalisés sur le Linpack avec un seul processus MPI (en GFlops).*

	Utilisation de quatre processus MPI.					
	ATLAS			MKL		
Problème	malloc	vhp	Gain (%)	malloc	vhp	Gain (%)
1000	13,14	13,02	-0,91	19,21	17,89	-6,87
2000	18,17	17,53	-3,52	23,63	23,54	-0,38
3000	21,08	20,81	-1,28	26,76	26,27	-1,83
6000	22,11	22,61	2,26	32,25	31,54	-2,2
10240	24,61	25,44	3,37	35,11	34,64	-1,34
17920	27,3			37,57	37,48	-0,24
19328	27,59			37,85	37,82	-0,08

Table C.2.: *Résultat des maximums obtenus pour les différents tests réalisés sur le Linpack avec quatre processus MPI (en GFlops).*

D. Configuration du code de MHD

```
Test problem number
12
Order
4
Transverse reconstruction
fluxes
Number of cells along X direction
400
Number of cells along Y direction
400
Final time
5.0
CFL
0.9
Remap method
centerPolyRec
Cbeta
1.0
Cnu
0.002
Ckappa
0.01
Ceta
0.001
Number of screen outputs
10
Number of solution outputs
0
```

Bibliographie

[1] David A.PATTERSON John L. HENNESY : *Computer Architecture : A Quantitative Approach.* Morgan Kaufmann, 3 édition, mai 2002.

[2] CEA - commissariat à l'énergie atomique. http ://www.cea.fr.

[3] INTEL : *Intel 64 and IA32 Architectures optimization reference manual*, mars 2009.

[4] Andrew S. Tanenbaum : *Modern Operating Systems.* Prentice Hall, 3 édition, décembre 2007.

[5] Daniel P. BOVET et Marco CESATI : *Understanding the Linux kernel.* O'REILLY, 3 édition, décembre 2005.

[6] Linux sources. http ://www.kernel.org/.

[7] linux-mm.org. http ://linux-mm.org.

[8] Juan Navarro, Sitaram Iyer, Peter Druschel et Alan Cox : Practical, transparent operating system support for superpages. *In OSDI '02 : Proceedings of the 5th symposium on Operating systems design and implementation*, pages 89–104, New York, NY, USA, 2002. ACM.

[9] Jim MAURO et Richard MC DOUGALL : *Solaris Internals, core kernel Architecture.* Prentice Hall PTR, novembre 2000.

[10] Ian WIENAND : *Transparent Large-Page Support for Itanium Linux.* Thèse de doctorat, The University of New South Wales, juillet 2008.

[11] John MARVIN : *Port of Shimizu superpages patch to ia64/x86-64*, octobre 2006. http ://lkml.org/lkml/2006/10/30/193.

[12] Jonathan CORBET, Alessandro RUBINI et Greg KROAH-HARTMAN : *Linux device drivers.* O'REILLY, 3 édition, mars 2005.

[13] Corbet : fault(). *lwm.net*, (242625), juillet 2007. http ://lwm.net/Articles/242625.

[14] Corbet : Faulting out populate(), nopfn() and nopage(). *lwm.net*, (203725), octobre 2006. http :///lwm.net/Articles/203725/.

[15] Marc PÉRACHE et Patrick CARRIBAULT : MPC - multiprocessors communications. http ://mpc.sourceforge.net/.

[16] National Science Foundation : BLAS - basic linear algebra subprograms. http ://www.netlib.org/blas/.

[17] National Science Foundation : LAPACK - linear algebra package. http ://www.netlib.org/lapack/.

[18] R. Clint Whaley et Antoine Petitet : ATLAS - automatically tuned linear algebra software. http ://math-atlas.sourceforge.net/errata.html.

[19] INTEL : Processors, juillet 2006. http ://www.intel.com/support/processors/sb/cs-023143.html.